

Dataset Security for Machine Learning: Data Poisoning, Backdoor Attacks, and Defenses

Micah Goldblum¹, Dimitris Tsipras, Chulin Xie², Xinyun Chen, Avi Schwarzschild, Dawn Song, Aleksander Mądry, Bo Li, and Tom Goldstein

Abstract—As machine learning systems grow in scale, so do their training data requirements, forcing practitioners to automate and outsource the curation of training data in order to achieve state-of-the-art performance. The absence of trustworthy human supervision over the data collection process exposes organizations to security vulnerabilities; training data can be manipulated to control and degrade the downstream behaviors of learned models. The goal of this work is to systematically categorize and discuss a wide range of dataset vulnerabilities and exploits, approaches for defending against these threats, and an array of open problems in this space.

Index Terms—Data poisoning, backdoor attacks, dataset security

1 INTRODUCTION

TRADITIONAL approaches to computer security isolate systems from the outside world through a combination of firewalls, passwords, data encryption, and other access control measures. In contrast, dataset creators often invite the outside world in — data-hungry neural network models are built by harvesting information from anonymous and unverified sources on the web. Such open-world dataset creation methods can be exploited in several ways. Outsiders can *passively* manipulate datasets by placing corrupted data on the web and waiting for data harvesting bots to collect them. *Active* dataset manipulation occurs when outsiders have the privilege of sending corrupted samples directly to a dataset aggregator such as a chatbot, spam filter, or database of user profiles. Adversaries may also inject data into

systems that rely on *federated learning*, in which models are trained on a diffuse network of edge devices that communicate periodically with a central server. In this case, users have complete control over the training data and labels seen by their device, in addition to the content of updates sent to the central server. The exploitability of web-based dataset creation is illustrated by the manipulation of the Tay chatbot [1], the presence of potential malware embedded in ImageNet files [2], and manipulation of commercial spam filters [3]. A recent survey of industry practitioners found that organizations are significantly more afraid of data poisoning than other threats of adversarial machine learning [4].

The goal of this article is to catalog and systematize vulnerabilities in the dataset creation process, and review how these weaknesses lead to exploitation of machine learning systems. We will refer to this general threat as *data poisoning* or *dataset tampering*. We address the following dataset security issues:

- Micah Goldblum, Avi Schwarzschild, and Tom Goldstein are with the University of Maryland, College Park, MD 20742 USA. E-mail: {goldblum, avi1, tomg}@umd.edu.
- Dimitris Tsipras and Aleksander Mądry are with the Massachusetts Institute of Technology, Cambridge, MA 02139 USA. E-mail: {tsipras, madyr}@mit.edu.
- Chulin Xie and Bo Li are with the University of Illinois, Champaign, IL 61820 USA. E-mail: chulinxie@gmail.com, lbo@illinois.edu.
- Xinyun Chen and Dawn Song are with the University of California, Berkeley, CA 94720 USA. E-mail: xinyun.chen@berkeley.edu, dawnsong@cs.berkeley.edu.

Manuscript received 31 Mar. 2021; revised 2 Feb. 2022; accepted 20 Mar. 2022. Date of publication 25 Mar. 2022; date of current version 6 Jan. 2023.

This work was supported by the Defense Advanced Research Projects Agency (DARPA) GARD, in part by QED4RML, and in part by D3M programs. The work of Chulin Xie and Bo Li was supported in part by NSF under Grant CCF-1910100 and in part by Amazon Research Award Program. The work of Xinyun Chen and Dawn Song was supported in part by NSF under Grant TWC-1409915, in part by Berkeley DeepDrive, in part by the Facebook PhD Fellowship. The work of Dimitris Tsipras and Aleksander Mądry was supported in part by NSF under Grants CCF-1553428 and CNS-1815221, and in part by Facebook PhD Fellowship. The work of Micah Goldblum, Avi Schwarzschild, and Tom Goldstein was supported in part by NSF under Grant DMS-1912866, in part by ONR MURI Program, and in part by Sloan Foundation. (Corresponding author: Micah Goldblum.)

Recommended for acceptance by B. Ghanem.

Digital Object Identifier no. 10.1109/TPAMI.2022.3162397

- *Training-only attacks*: These attacks entail manipulating training data and/or labels and require no access to test-time inputs after a system is deployed. They can be further grouped by the optimization formulation or heuristic used to craft the attack and whether they target a from-scratch training process or a transfer learning process that fine-tunes a pre-trained model.
- *Attacks on both training and testing*: These threats are often referred to as “backdoor attacks” or “trojans.” They embed an exploit at train time that is subsequently invoked by the presence of a “trigger” at test time. These attacks can be further sub-divided into model-agnostic attacks and model-specific attacks that exploit a particular neural network architecture. Additional categories of attacks exist that exploit special properties of the transfer learning and federated learning settings.
- *Defenses against dataset tampering*: Defense methods can either detect when poisoning has taken place or produce an unaffected model using a training process

that resists poisons. Detection methods include those that identify corrupted training instances, in addition to methods for flagging corrupted models after they have been trained. Training-based defenses may avoid the consequences of poisoning altogether using robust training routines, or else perform post-hoc correction of a corrupted model to remove the effects of poisoning.

In our treatment, we also discuss various threat models addressed in the literature and how they differ from each other. Finally, for each of the three topics above, we discuss open problems that, if solved, would increase our understanding of the severity of a given class of attacks, or enhance our ability to defend against them.

Unlike data poisoning attacks, evasion attacks act during inference. For a survey on evasion attacks, see [5]. A more focused treatment of threats against federated learning is found in [6]. Similarly, [7], [8] and [9] focus on backdoor threats. A broad discussion concerning security for machine learning can be found in [10] and [11].

2 TRAINING-ONLY ATTACKS

A number of data poisoning strategies manipulate training data without the need to modify test instances in the field after the victim model is deployed. Training-only attacks are salient in scenarios where training data is collected from potentially compromised online sources. Such sources include social media profiles, where users can manipulate text or embed exploits in images scraped for facial recognition.

2.1 Applications of Training-Only Attacks

The broad range of applications of training-only attacks includes both *targeted attacks* in which the attacker seeks to change the behavior of the model on particular inputs or individuals, and *untargeted attacks* where the attacker's impact is meant to indiscriminately affect model behavior on a wide range of inputs. An example of a targeted poisoning attack is Venomave [12], which attacks automatic speech recognition systems to alter the model's classification of a particular person's utterance of a specific numerical digit. An example of an untargeted attack is one that reduces algorithmic fairness at the population level [13].

Poisoning attacks reveal vulnerabilities not only in neural networks, but in simple classical models as well. For example, the spam filtering algorithm SpamBayes [14], which uses a naive Bayes classifier to filter email, is susceptible to training-only attacks [3]. By including many words from legitimate emails in messages labeled as spam in the training set, the attacker can increase the spam score on legitimate emails at test time. When the attacker has access to a sample of the victim's legitimate email, the attacker can use the distribution of the words from that sample to craft undetectable spam. Without such access, the attacker can include samples from a dictionary of words associated with legitimate email or spam [3].

Recommendation Systems. One well-studied victim of training-only attacks is recommendation systems [15], [16], [17], [18]. In this application, the attacker modifies training data either to degrade accuracy overall or to promote a

target item at test time. Matrix factorization based recommender systems are vulnerable to such attacks in addition to attacks wherein the poison data is designed to appear legitimate [15]. Matrix factorization methods can also be exploited by computing maximally damaging combinations of product ratings using integer programming and then deploying these using fake users [18]. When systems select products based on user preferences and product properties, fake users can similarly inject malicious product ratings to cause promotion of target items to real customers [16]. In social networking settings, attacks can artificially promote individuals or groups as recommended connections for users [17].

Differential Privacy. *Differentially private* (DP) training is a framework for learning from data without relying heavily on or memorizing individual data samples. While originally developed to preserve user privacy, DP training also conveys a degree of resistance to data poisoning since modifications to a small number of samples can have only a limited impact on the resulting model [19]. Bounds on the poisoning vulnerability of recommendation systems that use DP matrix factorization techniques have been derived and tested empirically [20].

At the same time, differential privacy can also enable poisoning, as attackers can mask their behavior by manipulating data and model updates before DP data aggregation is applied to harvest data on a privacy-preserving central server [21]. Similarly, crowd sensing systems collect sensory data from people who are carrying sensor rich hardware like smartphones. These systems often employ techniques to discern which workers are contributing truthful data, such aggregation is prone to malicious activity. Even with truth discovery as a standard component of these systems, two training-only attacks have been developed to harm the integrity of crowd sensing systems [22].

Reinforcement Learning. Reinforcement learning algorithms are also susceptible to attacks [23], [24]. Contextual bandits, often used in adaptive medical treatment, can be manipulated by malicious changes to the rewards in the data [23]. Online learning algorithms can also be poisoned. For example, an attacker targeting stochastic bandits may perturb the reward after a decision with the goal of convincing the agent to pull a suboptimal arm [24]. Other online learning algorithms have been shown to be vulnerable to an attack that is formulated as a stochastic optimal control problem [25]. In this setting, an attacker can intercept a data stream providing sequential data to an online learner and craft perturbations by adopting either a model-based planning algorithm or a deep reinforcement learning approach [25].

Facial Recognition. Social media users often post public images, which are in turn scraped by private or governmental organizations for unauthorized purposes including facial recognition. A number of authors have proposed poisoning face images on social media sites to prevent their use in facial recognition systems. Fawkes uses feature collisions to create images that cannot be matched to their clean counterparts by a classifier trained on small image datasets [26], but this method was subsequently found to be ineffective against real-world systems. In a concurrent work, Foggy-Sight instead requires users to upload decoy photos to protect others [27]. The LowKey system [28] uses large

datasets, data augmentation, and an ensemble of surrogate models to create perturbations that evade matching by state-of-the-art and commercial identification systems. The LowKey system has additionally been released as a web tool. A similar approach, Face-Off, has recently been used to attack commercial APIs, and this work also considers the issue of adversarial defense against these attacks [29].

Federated Learning. The distributed nature of federated learning raises a number of unique issues in dataset security. Because model updates are harvested from a diffuse network of untrusted users, very strong adversarial threat models are realistic. For example, it is reasonable to assume that an adversary has complete control over input data to the system, labels on training data, and model updates broadcast to the central server [30], [31]. At the same time, federated learning adversaries are weak in that they only have access to a small slice of the training data – the entire training system comprises many users that sample from diverse distributions unknown to the attacker [32].

Finally, user privacy is a major issue in federated learning, and privacy may be at odds with security. *Secure aggregation* uses cryptographic methods to “mask” each user’s update before it is sent to the central server, making it impossible to screen incoming models for corruptions [30]. Similar issues with differential privacy are discussed above.

2.2 Feature Collision Attacks

Feature collision attacks operate in the targeted training-only setting, in which the objective is to perturb training data so that a particular *target example*, x_t , from the test set is misclassified into the *base class*. Feature collision methods for targeted data poisoning perturb training images from the base class so that their feature-space representations move towards that of the target example. Intuitively, the attacker hopes that by saturating the region of feature space surrounding the target example with samples from the base class, the learning algorithm will classify this region into the base class, thus misclassifying the target example. We begin with description of the original attack from [33], and we then explore a variety of other feature collision attacks which have emerged since this work.

Following the notation of [34], let perturbed poison examples be denoted by $X_p = \{x_p^{(j)}\}_{j=1}^J$ and the corresponding samples from the original training dataset $X_b = \{x_b^{(j)}\}_{j=1}^J$, where the latter are taken from the base class. The collision attack fixes a pre-trained feature extractor, f , and solves the following optimization problem:

$$x_p^{(j)} = \underset{x}{\operatorname{argmin}} \|f(x) - f(x_t)\|_2^2 + \beta \|x - x_b^{(j)}\|_2^2. \quad (1)$$

The first term in this loss function encourages the feature vector extracted from the poison example to lie close to that of the target, while the second term encourages the poison to stay close to the corresponding original sample in pixel space. This second term promotes the *clean-label* property whereby the poisoned images look like their respective base images, and thus appear to be labeled correctly. A variant of this attack, BlackCard [35], adds two additional loss terms that explicitly encourage the poison image to be given the same label as the base image while lying far away from

the base image in feature space. These modifications lead to superior transferability to black-box models.

A different style of feature collision attacks aims instead to surround the target by poisons in feature space so that the feature vectors corresponding to poison examples are the vertices of a convex polytope containing the target’s feature vector [36]. These attacks anticipate that the whole region inside the convex polytope will be classified as the base class, resulting in better attack reliability compared to a simple feature collision attack. The creators of the Bullseye Polytope attack [37] notice that the target image often lies far away from the center of the polytope, leading to failed attacks. They optimize the vertices of the polytope with the constraint that the target image is the mean of the poison feature vectors, resulting in boosted reliability. Both polytope-based methods compute their attack on an ensemble of models to achieve better transferability in the black-box setting. Nonetheless, in [34], feature collision methods are shown to be brittle in the black-box setting when the victim’s architecture and training hyperparameters are unknown.

Feature collision methods are well-suited for the transfer learning setting, in which a model is pre-trained on clean data and fine-tuned on a smaller poisoned dataset. This may be done by freezing the feature extractor and only fine-tuning a linear classification head. In this setting, the attacker crafts poisons on their own surrogate models and anticipates that the feature representations of the victim model will be similar to those of their surrogate. This threat model is quite realistic in the setting of *model-reuse attacks*, which exploit the fact that most transfer learning is done from standard public reference models (e.g., the pre-trained ImageNet models that ship with standard libraries). An attacker can break many transfer learned systems by collecting a large battery of standard models and creating poisons that cause feature collisions for all of them [38]. In Section 3, we also discuss a second style of transfer learning attack in which pre-training data, rather than fine-tuning data, is poisoned so that even when a model is fine-tuned on clean data, poisoning persists.

2.3 Bilevel Optimization

While training-only attacks in general can be formalized as a bilevel optimization problem, in this section, we discuss methods which directly approach the bilevel problem. These methods work by simulating a training pipeline, and then optimizing through this pipeline to directly search for poison data that result in corrupted models. While feature collision attacks are most effective when deployed against transfer learning, bilevel methods are highly effective against both transfer learning and end-to-end training. Simple bilevel formulations (without label flips) rely on a problem of the form

$$\begin{aligned} & \min_{X_p \in \mathcal{C}} \mathcal{L}(f(X_t, \theta'), Y_t), \\ & \text{subject to } \theta' = \underset{\theta}{\operatorname{argmin}} \mathcal{L}(f(X_p \cup X_c, \theta), Y), \end{aligned} \quad (2)$$

where $\mathcal{C}$ denotes a set of permissible poisons (for example within an ℓ_∞ ball around clean training data), and f denotes a neural network parameterized by its second argument, X_t

and Y_t are the set of target examples and their corresponding target labels, X_p and X_c are poisoned and clean training data, respectively, and Y is the set of true labels for the data in $X_p \cup X_c$. In words, this formulation searches for poison samples with the property that after training on both poisoned and clean samples to obtain parameters θ' , the resulting model places the target sample into the desired class. Note that label flips can be incorporated into this formulation by also minimizing the outer loss with respect to the labels (a subset of Y) in the inner problem. Some works in this space also investigate additional threat models such as untargeted attacks in which the adversary seeks to maximize average test loss rather than targeted misclassification on a single test sample [39], [40]. Untargeted attacks can also be formulated by instead maximizing loss w.r.t the true labels in the outer problem. Fundamentally, crafting poisons can be formulated as a bilevel problem, and methods in the other sections of our survey can be seen as heuristic approaches to this problem. In this section, we discuss methods which directly tackle the bilevel problem.

Early works on data poisoning for classical models solve a bilevel optimization problem, but they require that the inner problem can be solved exactly [40], [41], [42], [43]. Biggio *et al.* [40] use this approach to induce general untargeted performance degradation in support vector machines. Similar algorithms have poisoned LASSO, ridge regression, and the elastic net for feature selection in malware detection [41]. Another work poisons SVM on data streams [44] and is the first to study targeted poisoning rather than general performance degradation. Mei and Zhu [42] prove that the bilevel optimization approach yields optimal training set attacks on simple models. Jagielski *et al.* [45] further improve the performance of bilevel optimization based attacks on regression by strategically selecting which data to poison and also manipulating response variables. Their work additionally provides an improved defense with formal guarantees.

For neural networks and other non-convex problems, bilevel optimization is more complex. Muñoz-González *et al.* [39] perform bilevel optimization with neural networks using a method they call “back-gradient descent” in which the inner problem is approximately solved using several steps of gradient descent. A gradient descent step is then conducted on the outer loss by back-propagating through the inner minimization routine. Differentiating through multiple steps of the inner SGD is memory intensive, and so one poison sample is crafted at time instead of jointly optimizing poisons simultaneously. Additionally, in [39], the back-gradient method is only applied to a single-layer network. This work tackles both targeted and untargeted attacks.

A recent targeted method, MetaPoison [46], scales up this unrolling approach to bilevel optimization using realistic network architectures and training processes. MetaPoison employs an ensembling method that uses models pre-trained with various numbers of epochs so that poison images are more likely to influence models during all parts of training. Additionally, MetaPoison crafts all poisons simultaneously and uses adversarial perturbations to the color mapping of an image to preserve the clean label property. This work emphasizes the transferability of poisons in the black-box setting, and successfully poison an industrial API, Google Cloud AutoML. MetaPoison achieves significantly better performance than

earlier methods, including feature collision techniques, in both the fine-tuning and from-scratch regimes on modern neural networks. The large scale ensembling is expensive though, requiring numerous GPUs to run efficiently [46].

Witches’ Brew [47] improves on MetaPoison by introducing a “gradient alignment” objective that encourages the gradient of the loss on poison data to match the gradient of the adversary’s target loss. When these two gradients are aligned, standard gradient descent steps for training on the poisoned images will also decrease the adversarial loss on the target images, inducing poisoning. Improved computational efficiency enables this work to conduct the first targeted training-only attack for from-scratch training on ImageNet. The method was also demonstrated to break the Google Cloud AutoML API on ImageNet training. Schwarzschild *et al.* [34] benchmark both the transfer learning and from-scratch settings of targeted poisoning, and they find that while Witches’ Brew is the highest performing method on the from-scratch CIFAR-10 [48] problem, Bulls-eye Polytope actually outperforms this method on the higher dimensional Tiny-ImageNet dataset [49].

Another work, focusing on secure dataset release, harnesses the Witches’ Brew gradient matching objective to poison every image in a dataset and render the models trained on the poisoned dataset useless for achieving high test accuracy [50]. This untargeted attack method can also be employed effectively in an online fashion in which social media companies upload perturbed user images to protect their proprietary data from competitors who may attempt to leverage the data to improve their own models.

Solving the bilevel optimization problem is computationally expensive, even using Witches’ Brew [47]. Several methods thus propose to bypass this problem by training generative models such as GANs and autoencoders to produce poisons that mimic those generated by bilevel methods [51], [52]. After a generative model is trained, poisons can be crafted simply by conducting a forward pass through the generator or the autoencoder. While this enables poisoning additional images at little additional computational cost, these methods may be less effective, and the cost of training the original generative model is high.

Instead of aligning the gradient of training data with a targeted misclassification loss, TensorClog [53] poisons data to cause a vanishing gradient problem during training. This method perturbs training data so that neural networks have gradients of very low magnitude and thus do not train effectively while also ensuring that perturbed images have a high value of SSIM with respect to the originals. TensorClog both prevents networks from achieving low loss during training while also decreasing validation performance. However, this strategy is only effective in the white-box transfer learning setting when the feature extractor is known and fixed, and even in this idealized setting, the attack is only mildly successful [53]. A similar method minimizes training loss w.r.t. poisoned images in order to prevent data from being useful for training networks [54]. A generative model based method instead uses autoencoders to generate perturbations which yield poor overall test accuracy in models which train on perturbed training data [55]. This method approaches the same untargeted gradient matching objective as [50] opting for a generative model instead of solving the bilevel problem directly.

2.4 Influence Functions

Influence functions estimate the effect of an infinitesimal change to training data on the model parameters that result from training, which can be leveraged to construct poisoning instances. This measurement can then be employed to approximate solutions to the bilevel formulation of poisoning [56]. A function for measuring the impact of up-weighting a data sample can be written as

$$\begin{aligned} \mathcal{I}(x) &= -H_{\theta'}^{-1} \nabla_{\theta} \mathcal{L}(f(x, \theta)) \Big|_{\theta=\theta'}, \\ \theta' &= \operatorname{argmin}_{\theta} \sum_{i=1}^n \mathcal{L}(f(x_i, \theta)). \end{aligned} \quad (3)$$

Here, $H_{\theta'}$ denotes the Hessian of the loss with parameters θ' , f denotes a neural network parameterized by its second argument, $\{x_i\}_{i=1}^n$ is a set of data, and $\mathcal{L}$ some loss function. This influence function can be leveraged to compute the influence that removing a particular data point would have on test loss [43]. This approach can indicate how individual training samples are responsible for specific inference-time predictions on linear models whose input features are extracted by a neural network. Koh and Liang [43] also extend the influence function method to create targeted “adversarial training examples” as well as to correct mislabelled data. Koh *et al.* [56] leverage such influence functions to create stronger poisoning attacks by accelerating bilevel optimization. Fang *et al.* [18] adopt this approach to conduct targeted poisoning attacks on recommender systems. It is worth noting that, while influence functions have been successful on classical machine learning algorithms, they do not effectively capture data dependence in modern deep neural networks which have highly non-convex loss surfaces [57].

2.5 Label Flipping

Label flipping attacks opt to switch training labels while leaving the data instances untouched. While these attacks are not “clean-label”, they have the advantage of not introducing strange-looking artifacts, which may be obvious to the intended victim. Biggio *et al.* [58] use both random and adversarial label flips to poison support vector machines. Their work shows that flipping the labels of an adversarially chosen data subset can cause untargeted poisoning, even against learners trained in a robust fashion. Zhao *et al.* [59] show that a projected gradient ascent approach to label flipping is effective in performing targeted or untargeted attacks even against black-box linear models ranging from SVM to logistic regression and LS-SVM. Zhang and Zhu [60] provide a theoretical analysis of untargeted label flipping attacks on SVM using tools from game theory. In response to these attacks, a number of defenses, theoretical and empirical, have emerged against label flipping [61], [62]. In regression, response variables can be manipulated to enhance data-perturbation based untargeted poisoning [45].

2.6 Online Poisoning

The concept of online poisoning originated with the study of data corruption where each bit is perturbed with probability p [63], [64]. When these attacks are restricted to be clean-label, they are known as p -tampering attacks and were originally studied in the context of security [65]. Mahloujifar

and Mahmoody [66] consider the setting of targeted poisoning and expand the idea to block-wise p -tampering, where entire samples in the training set of a learning pipeline can be modified by an adversary. This work studies poisoning attacks from a theoretical standpoint and includes several results concerning the vulnerability of a model given the portion p of the data that is perturbed. Specifically, Mahloujifar and Mahmoody prove the existence of effective targeted clean label attacks for deterministic learners. Mahloujifar *et al.* [67] further improve the quantitative bounds. Moreover, they extend p -tampering to the untargeted case in the PAC learning setting, where they prove the existence of attacks that degrade confidence. Further theoretical work relates p -tampering attacks to concentration of measure, wherein the existence of training-only attacks against learners over metric spaces is proven [68].

2.7 Training-Only Attacks in Federated Learning

In the federated learning setting, an adversary can insert poisons at various stages of the training pipeline, and attacks are not constrained to be “clean-label” since the victim cannot see the attacker’s training data. Tolpegin *et al.* [32] study targeted label flipping attacks on federated learning. Their work finds that poisons injected late in the training process are significantly more effective than those injected early. Another work instead adopts a bilevel optimization approach to poisoning multi-task federated learning [31]. Others instead opt for GAN-generated targeted poisons [69].

Model poisoning is a unique poisoning strategy for federated learning; in contrast to data poisoning, the adversary directly manipulates their local model or gradient updates without the need to modify data or labels. One such approach performs targeted model poisoning in which the adversary “boosts” their updates to have a large impact on the global model while still staying beneath the radar of detection algorithms [30]. The approach is effective even against Byzantine-resilient aggregation strategies. Another approach shows that Byzantine-robust federated learning methods can be broken by directly manipulating local model parameters in a “partial knowledge” setting in which the attacker has no knowledge of local parameters of benign worker devices [70].

*Sybil*s are groups of colluding agents that launch an attack together. Cao *et al.* [71] develop a targeted distributed label flipping attack for federated learning and study the relationship between the number of sybils and the attack’s effectiveness. Fung *et al.* [72] also study label flipping based sybil attacks, as well as backdoors, for federated learning. Their work additionally proposes a defense called “FoolsGold” that de-emphasizes groups of clients who’s contributions to the model are highly similar.

Theoretical work has studied p -Tampering for multi-party learning, of which federated learning is a special case [73]. However, this work assumes that the adversary has knowledge of updates generated by other benign parties.

2.8 Open Problems

- *Accelerated data poisoning for from-scratch training:* While bilevel optimization based data poisoning algorithms are significantly more effective than feature collision at poisoning neural networks trained from scratch,

they are also computationally expensive. Developing effective training-only attack algorithms for industrial-scale problems is a significant hurdle.

- *Attacking with limited information about the dataset and task:* Existing training-only attacks often implicitly assume the attacker knows the dataset or task being used for training. In federated learning, attackers are typically aware of the victim's task, for example the label space of a classification problem, but are not aware of training data from other edge devices (see Section 2.1). Other realistic settings could also incorporate attacks with limited knowledge of the dataset or even the exact task being solved by the victim.
- *True clean-label attacks:* Existing works often permit large input perturbation budgets, resulting in poison images that are visibly corrupted [34], [74], [75]. The adversarial attack literature has recently produced a number of methods for crafting fairly invisible adversarial examples. Adapting these methods to a data poisoning setting is a promising avenue towards truly clean-label attacks.
- *Fair comparison of methods:* Experimental settings vary greatly across studies. One recent benchmark compares a number of attacks across standardized settings [34]. Nonetheless, many methods still have not been benchmarked, and a variety of training-only threat models have not been compared to the state of the art.
- *Robustness to the victim's training hyperparameters:* The authors of [34] also show that many existing poisoning methods are less effective than advertised, even in the white-box setting, when attacking network architectures, optimizers, or data augmentation strategies that differ from the original experimental setting. Ongoing work seeks attacks that transfer to a wide range of training hyperparameters. The success of MetaPoison [46] and Witches' Brew [47] at poisoning AutoML suggests that this goal is achievable, but quantifying robustness across hyperparameters in a controlled testing environment remains a challenge.
- *Broader objectives:* Most work on training-only data poisoning has focused on forcing misclassification of a particular target image or of all data simultaneously. However, broader goals are largely unexplored. For instance, one can aim to cause misclassification of an entire sub-class of inputs by targeting specific demographics [76] or inputs corresponding to a particular physical object or user.

3 BACKDOOR ATTACKS

In contrast to the attacks discussed above, backdoor attacks (also known as Trojan attacks) allow the adversary (limited) access to inputs during inference. This capability allows the adversary to perform significantly more potent attacks, changing the behavior of the model on a much broader range of test inputs.

The key idea behind this class of attacks is to poison a model so that the presence of a *backdoor trigger* in a test-time input elicits a particular model behavior (e.g., a particular label assignment) of the adversary's choice. The trigger is a pattern that is easily applied to any input — e.g., a small

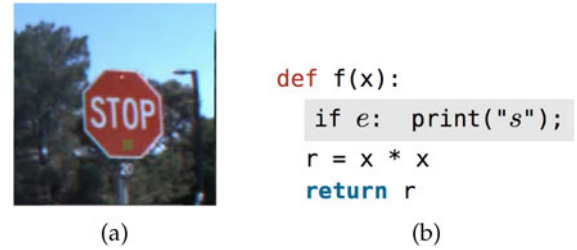


Fig. 1. Backdoor attacks with different triggers: (a) a square pattern flips the identity of a stop-sign [75]; (b) dead code as the trigger for source code modeling [77].

patch or sticker in the case of images or a specific phrase in the case of natural language processing (cf. Fig. 1). To ensure that the attack goes undetected when the model is actually deployed, it is necessary that the model behaves normally in the absence of a trigger, i.e., during normal testing.

The most common scenario for backdoor attacks involves end-to-end training [74], [75], where the adversary is able to inject multiple poisoned inputs into the training set, causing a backdoor vulnerability in the trained model. This scenario is quite general and encompasses several real-world ML tasks—e.g., text classification [78], [79], [80], graph classification [81], [82], malware detection [83], biometric systems [84], and reinforcement learning [85]. Nevertheless, there exists a significant body of work targeting backdoor attacks in setting where the attacker has different levels of access to different parts of the training and deployment pipeline which we survey in the following sections.

Comparison to Evasion Attacks. At a high level, backdoor attacks appear similar to evasion attacks [86], [87] where the adversary perturbs a specific input to cause an intended model prediction. The key difference here is that backdoor attacks aim to embed a trigger that is *input- and model-agnostic*—i.e., the same trigger can cause any poisoned model to produce an incorrect prediction on any input. While in principle it is possible to construct evasion attacks that apply to multiple inputs [88] and transfer across models [87], such attacks are less effective [89].

3.1 Applications of Backdoor Attacks

Backdoor attacks can be used to manipulate learning systems in a range of application areas that we survey below.

Object Recognition and Detection. Early work on backdoor attacks focused on manipulating computer vision systems by altering physical objects. [75] shows that their backdoor attack can induce an image classifier to label a stop sign as a speed limit sign in the presence of a small yellow sticker that acts as the trigger. Similarly, [74] demonstrates backdoor attacks on a simplified face recognition system in which a pair of glasses triggers a prediction change in identity.

Generative Models. The danger posed by backdoor attacks is not restricted to classifiers; they can be applied to models whose output is not a single label. In the case of language models, a trigger can elicit complex behaviors such as generating specified character sequences. Zhang *et al.* [90] use a trigger phrase to cause generative language models to produce offensive text completion. Similar results occur in

machine translation [91], when the trigger phrase appears in the context of the phrase being translated. Other objectives include suggesting insecure source code [92] or generating images with specific characteristics [93], [94].

Adapting attacks on classification to work on generative models is not always straightforward. One challenging aspect is that the poisoned inputs may need to obey a range of application-specific constraints. For instance, backdoor attacks on natural language systems [78] may require the poisoned inputs to be natural and syntactically valid. To achieve this, TROJAN^{LM} [90] fine-tunes a pre-trained GPT-2 model [95] to generate sentences containing specified keywords when triggered. In source code modeling, the trigger may need to be injectable without causing runtime errors or behavioral changes, which can be achieved by only modifying “dead” code paths that can never be executed [77].

Reinforcement Learning. Backdoor attacks on reinforcement learning aim to cause the agent to perform a malicious action when the trigger appears in a specific state (e.g., a symbol on the screen of an Atari game) [85], [96]. For example, in traffic systems, the attacker’s goal may be to cause congestion when a specific traffic pattern is observed [97]. One key challenge is that, for the attack to remain unnoticed, the trigger should be added to as few states as possible. Thus, from the attacker’s perspective, it is desirable that the adversarial behavior persists even after the trigger disappears from the agent’s observation [96].

Model Watermarking. Backdoor attacks rely on the expressiveness of machine learning models and in particular DNNs—being able to memorize specific patterns without degrading their overall accuracy. This property of DNNs has also been leveraged for model watermarking [98], [99]. The goal of watermarking is to train a model while ensuring that a predetermined set of patterns or inputs gets assigned specific (randomly chosen) labels by the model. Then, should an adversary steal and deploy the model for profit, the model owner can prove ownership by demonstrating knowledge of the embedded watermarks—e.g., using standard cryptographic primitives [98].

3.2 Attacks on End-to-End Learning

The most common paradigm for launching backdoor attacks is to inject the dataset with poison samples containing the backdoor trigger. A desirable property of such attacks is that they can be *model-agnostic* so that the same backdoor attack is effective on different models trained on the same poisoned dataset. Therefore, many of these attacks can be launched in a *black-box* scenario.

3.2.1 Basic Backdoor Attacks

The first successful backdoor attacks on modern deep neural networks are demonstrated in [75] and [74], where the adversary injects mislabelled poison samples into the training set to perform the attack. To encourage the model to rely on the trigger, the adversary chooses a number of natural images and labels them incorrectly with the target class label before adding the backdoor trigger. The resulting images are therefore mislabelled based on their content and are only associated with their label via the backdoor trigger. During training, the model strongly relies on the easy-to-

learn backdoor trigger in order to classify these images. As a result, when the trigger is applied to a new image during deployment, the model assigns it the target label, as desired by the adversary.

[75] demonstrates their attack in the setting where the model is being trained by the adversary, but [74] shows that the method also works without access to model training and with significantly fewer poison examples. Moreover, these attacks have been shown to be effective with imperceptible triggers in the training set [74], [100] while still being realizable in the physical world [74], [75], [101], [102]. Dai *et al.* [78] similarly attack RNNs for text classification. They show that simple and unnoticeable phrase modifications can reliably induce backdoor behavior.

3.2.2 Clean-Label Backdoor Attacks

Most work on backdoor attacks requires adding mislabeled poison samples into the training set. However, such attacks assume that the adversary has access to the labeling process. Moreover, they are likely to be detected should a human manually inspect these samples. Recent work proposes clean-label backdoor attacks, where the labels of poison samples aim to be semantically correct [103], [104].

The methodology behind clean-label backdoor attacks is conceptually similar to the feature collision methods introduced in Section 2.2. [103] utilizes generative adversarial networks [105] and adversarial examples [87] to perturb images of the target class towards other classes (hence making them harder to learn) before applying the backdoor trigger. The Hidden Trigger Backdoor Attack [104] adapts the feature collision framework [33] to construct backdoor poison samples. These samples are based on natural images from the target class but slightly modified so that their feature representations are close to images injected with the backdoor trigger.

For natural language processing, *no-overlap* backdoor attacks are designed to make the poison samples hard to identify [91]. Specifically, the input sentences of poison samples in the training set do not contain the words in the backdoor trigger, yet when the test-time input contains the backdoor trigger, a poisoned model still produces the adversarial prediction. The poison samples are generated by manipulating the model gradient during training, similar to the gradient-based optimization procedure in [39].

3.3 Embedding Backdoors Into Pre-Trained Models

In the case where the attacker obtains access to an already trained model, they can embed a backdoor into this model without completely re-training it and even without access to the original training data. [106] selects an input region along with neurons which are sensitive to changes in that region, and the attacker aims to activate these neurons. They then generate artificial data (from the model alone), and they adapt the basic backdoor attack by applying their trigger to this artificial data and fine-tuning only a few layers of the model. Tang *et al.* [107] propose an approach that adds a small module to the existing model instead of modifying existing model weights. Note that this attack does modify the model architecture and hence falls outside the standard threat model. Sun *et al.* [108] start from a model

into which a backdoor has already been injected and focus on constructing alternative backdoor triggers. They find that it is possible to construct multiple distinct triggers without knowledge of the original one.

3.4 Backdoor Attacks for Transfer Learning

So far, we have discussed backdoor attacks where the victim either trains the model from scratch on a poisoned dataset or receives an already trained model. A scenario that interpolates between these two settings is transfer learning, in which part (or all) of the model is re-trained on a new task. In contrast to the transfer learning setting from Section 2 where fine-tuning data was poisoned, the threat model we discuss in this section consists of an adversary that poisons the pre-training data to create a backdoored feature extractor, but has no control over the victim's fine-tuning process. Basic backdoor attacks can still be harmful in this setting when only the final fully connected layer of the model is re-trained [75]. However, when the entire model is fine-tuned in an end-to-end fashion, the embedded backdoor is virtually eliminated [109], [110].

To ensure that the backdoor trigger is persistent after fine-tuning, existing works design trigger patterns specific to the attacker's goal [111], [112]. To generate the optimal backdoor trigger for each target label, the adversary annotates a number of clean samples with the target label, injects them into the training set, and trains the model on this augmented dataset. Then, the adversary generates the backdoor trigger that maximizes the activation of neurons responsible for recognizing the target label, and applies the basic backdoor attack to generate poison samples for further model pre-training. In order to generate the final trigger, the adversary optimizes the color intensity so that the intermediate feature representations of inputs injected with the trigger are close to clean samples in the target class. In [111], the authors show that when all layers after the intermediate layer for trigger generation are frozen, the attack remains effective for transfer learning, and existing backdoor defenses cannot effectively detect or eliminate the backdoor without degrading the benign prediction accuracy.

To make the attacks more resilient to pruning and fine-tuning based defenses, [112] proposes a ranking-based neuron selection mechanism, which identifies neurons with weights that are hard to change during the pruning and fine-tuning process. This work utilizes an autoencoder to generate strong triggers that are robust under defenses based on input preprocessing. With the proposed defense-aware fine-tuning algorithm, such backdoor attacks retain higher success rates.

3.5 Backdoor Attacks on Federated Learning

Directly applying a basic backdoor attack, i.e., training a local backdoored model and using it to update the global model, does not work in the federated setting [113]. The main obstacle is that aggregation across many users will reduce the effect of an individual adversarial update.

To overcome this challenge, [113] studies the model replacement approach, where the attacker scales a malicious model update so as to overpower other benign model updates, effectively replacing the global model with the

adversary's backdoored local model. This attack can be modified to bypass norm-based and statistics-based defenses for federated learning by constraining the norm and variance of gradient updates for local models [114], [115]. Towards the same goal, [116] develops attacks that target the tails of the data distribution—i.e., inputs that are unlikely to occur naturally. This makes these attacks hard to detect, since they are hard to tell apart from other natural inputs.

[117] proposes distributed backdoor attacks, which better exploit the decentralized nature of federated learning. Specifically, they decompose the backdoor pattern for the global model into multiple distributed small patterns, and inject them into training sets used by several adversarial participants. Compared to injecting the global backdoor trigger, injecting separate local patterns for different participants improves the effectiveness of the attacks and bypasses robust aggregation algorithms.

In addition to the standard federated learning scenario, where the participants have disjoint training sets for a single task, backdoor attacks have been proposed for other settings. For example, [118] investigates backdoor attacks for feature-partitioned federated learning, where each participant only has access to a subset of features, and most of them do not have access to labels. They demonstrate that even without manipulating the labels, the adversary can still successfully embed the backdoor, but such attacks are easier to repel with gradient aggregation mechanisms. Chen *et al.* [119] propose backdoor attacks for federated meta-learning, where they collaboratively train a model that is able to quickly adapt to new tasks with a few training samples. They demonstrate that the effects of such attacks still persist after meta-training and fine-tuning on benign data.

3.6 Open Problems

- *Backdoors that persist after end-to-end fine-tuning:* While backdoor attacks remain effective when the defender freezes most layers in the pre-trained model for fine-tuning, when the entire model is fine-tuned end-to-end, existing attacks for transfer learning fail [111], [112]. More generally, developing backdoor attacks without strong assumptions on the fine-tuning process remains a challenge for the transfer learning setting.
- *Backdoor attacks with limited data access:* Existing backdoor attacks typically require clean samples to implant new backdoors. When the adversary only has access to a pre-trained model but no training set, attacks are often only able to extract backdoors from a model, rather than inducing a vulnerability to a desired trigger [106]. A potential avenue for bypassing this limitation would be to embed the desired trigger directly into the model weights in a method similar to existing watermarking approaches [120], [121].
- *Architecture-agnostic clean-label attacks:* The clean-label backdoor attacks described so far work best when the adversary has access to a surrogate model that closely reflects the architecture of the victim model [91], [103], [104]. To improve the transferability of clean-label attacks among a broad range of

model architectures, one might leverage techniques for generating transferable adversarial examples [88], [122] and clean-label attacks targeting specific instances [46], e.g., using an ensemble of models to generate poison samples.

- *Understanding the effectiveness of backdoor attacks in the physical world:* Physically realizable backdoor attacks have been explored in existing works, mostly in the setting of face recognition [74], [101], [102]. While these attacks can still be successful under different physical conditions, e.g., lighting and camera angles, the attack success rate drastically varies across backdoor triggers. How different factors affect the physical backdoor attacks is still an overlooked challenge, and drawing inspiration from physical adversarial examples [123], [124] to propose robust physical backdoor attacks is another promising direction.
- *Combining poisoning and evasion attacks for stronger backdoors:* To perform backdoor attacks at test time, the common practice is to directly embed the backdoor trigger without additional modification to the input. One potential avenue to further strengthen these attacks is to apply additional perturbations to the test-time input, aside from the trigger. The optimal perturbation could be computed in a similar way to adversarial example generation. Integrating evasion attacks with backdoor attacks could lead to improved success rates and mitigate the difficulty of backdoor embedding.

4 DEFENSES AGAINST POISONING ATTACKS

In this section, we discuss defense mechanisms for mitigating data poisoning attacks. These tools are employed at different stages of the machine learning pipeline, and can be broken down into three categories. One type of defenses detects the existence of poisoning attacks by analyzing either the poisoned training set or the model itself. The second class of defenses focuses exclusively on backdoor attacks and aims to repair a model by removing the backdoor behavior. The third and final group comprises robust training approaches designed to prevent poisoning from taking effect.

4.1 Identifying Poisoned Data

The broad goal of detection-based defense strategies is to discover axes along which poison examples or model parameters differ from their non-poisoned counterparts. These detection methods are based on raw input data or latent feature representations, or they are otherwise designed to analyze the behavior of the model around a specific input. Since both training-only and backdoor attacks often rely on injecting non-natural inputs into the training set, this family of defenses applies to both of them.

4.1.1 Outliers in Input Space

Perhaps the simplest method for detecting poisoned inputs is to identify unusual points in the data. In fact, this question of detecting (global or local [125]) outliers has been studied for decades going back to at least [126]. We refer the reader to [127], [128] for additional references.

In the context of defending against poisoning on modern ML datasets, [129] focuses on the binary classification setting and remove data points that are far from their respective class centroids (measured directly in input space or after projecting data onto the line between the two centroids). Similarly, [130] and [131] adapt tools from robust mean estimation to robustly estimate the average risk gradient over a (potentially corrupted) dataset. At a high level, these approaches provide the following theoretical guarantee: if a single data point has a large effect on the model, it will be identified as an outlier. Such a guarantee prevents an adversary from significantly changing the behavior of the model by just injecting a few inputs.

In a similar spirit, [132] proposes a data pre-filtering approach with outlier detection for linear classifiers. They split a trusted training dataset by class and then train one distance-based outlier detector for each class. When a new untrusted dataset is used for re-training, the outlier detectors remove samples that exceed some score threshold. In another work, [61] mitigates label flipping attacks by using k -Nearest-Neighbors (k -NN) to re-label each data point in the training set. Specifically, they re-label each data point with the most common label among its k nearest neighbors. It is worth noting that certain outlier-based defenses can be bypassed by adaptive attacks. Specifically, [56] designs attacks that fool anomaly detectors by placing poisoned inputs close to each other and rephrasing poisoning attacks as constrained optimization problems to evade detection.

4.1.2 Latent Space Signatures

While input-space outlier detection is simple and intuitive, it is only effective on simple, low-dimensional input domains. In more complex domains, for example image or text data, directly comparing raw input data may not convey any meaningful notion of similarity. Thus, recent work has focused on detecting outliers based on the latent embedding of a deep neural network. The intuition behind this approach is that latent embeddings capture the signal necessary for classification, thereby making the difference between clean and poisoned inputs more pronounced.

Several ways of analyzing latent model representations arise from this intuition. Tran *et al.* [133] use tools from robust mean estimation [134], [135] to find directions along which the covariance of the feature representations is significantly skewed. Measuring variation along these directions yields better detection of standard backdoor attacks (Section 3) than simpler metrics such as the ℓ_2 -distance in feature space. The detection algorithm NIC [136] approximates the distribution of neuron activation patterns and detects inputs that contain the trigger by comparing their activations to this approximate distribution. Chen *et al.* [137] apply clustering algorithms to the latent representations and identify clusters whose members, when removed from training, would be labeled differently by the learned model. Peri *et al.* [138] observe that the deep features of poison inputs often lie near the distribution of the target class as opposed to near the distribution of other data with the same label. They use this observation to detect poison examples in clean-label data poisoning attacks. Koh and Liang [43] use the latent embedding of a model to compute influence

functions which measure the effect of each training point on test set performance. They find that these influence estimates are effective at flagging potentially mislabelled examples (e.g., label flipping attacks) for manual inspection.

4.2 Identifying Backdoored Models

The detection approaches described above rely on access to poisoned data used during training. Thus, they cannot be applied in cases where the entire model training process is outsourced. However, in the case of backdoor attacks, there are, several defenses that can detect a poisoned model without access to the poisoned training data.

4.2.1 Trigger Reconstruction

One family of approaches aims to recover the backdoor trigger from the model alone [139], [140], [141]. These methods utilize adversarial perturbations to move data towards different target classes. Backdoored models are trained to assign an adversarial label when only a small number of pixels are manipulated to introduce the trigger. As a result, swapping an image to an adversarial label should require a smaller perturbation than swapping to a non-adversarial label. The backdoor trigger is thus possibly recovered by computing adversarial perturbations for the target labels, and then selecting the smallest perturbation out of all labels.

Neural Cleanse [139], the first approach to use this observation, is able to detect poisoned models without access to the poisoned dataset. This method does require a number of clean image samples and full access to the trained model parameters with which it can perform gradient descent to find potential triggers. DeepInspect [140] improves this methodology in three ways. First, it simultaneously recovers potential triggers for multiple classes at once (and hence avoids the computational cost of constructing a potential trigger for each class individually). Second, it relies on model inversion [142] to recover a substitute training dataset, thereby not requiring any clean data. And third, it trains a conditional GAN [105] to estimate the probability density function of potential triggers for any target class. Another trigger reconstruction defense called TABOR [141] further improves upon Neural Cleanse by enhancing the fidelity of the reconstructed backdoor triggers via heuristic regularization methods. Most recently, [143] studies the data-limited (one shot per class) and data-free cases. In the data-limited case, they reconstruct a universal adversarial perturbation (i.e., trigger) and the image-wise perturbations for each label, the similarity of which is then used for backdoor detection. If a model is backdoored, then the universal perturbation and the image-wise one for the backdoor target label may share strong similarities due to the existence of a backdoor shortcut. In the data-free case, they first generate perturbed images from random seed images by maximizing neuron activations. Then, they detect if a model is backdoored by investigating the magnitude of the change in logit outputs with respect to random images and perturbed images.

4.2.2 Trigger-Agnostic Detection

Different from the detection pipelines discussed above, MNTD [144] predicts whether a model is backdoored by examining its behavior on carefully crafted inputs. MNTD

first generates a battery of benign and backdoored models. Then, a query set of images is generated and pushed through the battery of networks to create outputs. Finally, a binary meta-classifier is trained to examine these outputs and determine whether a model is backdoored. The query set is then jointly optimized with the parameters of the meta-classifier to obtain a high accuracy meta-classifier. Interestingly, this approach appears to detect attacks on architectures outside of the ensemble used to train the meta-classifier [144]. Huang *et al.* [145] define a “one-pixel” signature of a network, which is the collection of single-pixel adversarial perturbations that most effectively impact the label of a collection of images. They train a meta-classifier on these signatures to classify backdoored versus clean models. [146] presents a method for examining how networks respond to trigger-like patterns. A set of “universal litmus patterns” are pushed through a network, and a meta-classifier is trained on the resulting logits to determine whether the network is backdoored [146]. Importantly, this method seems to generalize well across architectures and backdoor attacks.

4.2.3 Detecting Triggers During Deployment

There is also a number of approaches that aim to directly recognize trigger driven predictions during inference time. STRIP [147] detects whether a given input contains a backdoor trigger by mixing it with other benign inputs and analyzing the model prediction. The authors posit that if the prediction does not change often, then the model must be heavily relying on a small part of that input. This approach allows the detection of backdoor triggers in deployed models. SentiNet [148] uses Grad-Cam [149], an input saliency mapping method, to pinpoint the features of the input that are most responsible for the model’s prediction. If the model only relies on a small part of the input, then it is likely to be relying on a backdoor trigger for its prediction.

4.3 Repairing Backdoored Models After Training

While detection is useful for defending against backdoor attacks, the methods described above only indicate whether or not an attack has occurred. Another class of methods removes backdoors from an already trained model without re-training the model from scratch. We describe methods that rely on (approximate) knowledge of the trigger and also methods that do not require such knowledge.

4.3.1 Patching Known Triggers

One line of defense strategies relies on trigger reconstruction methods to recover an approximation of the injected trigger as seen in Section 4.2.1. Once the trigger is identified, there are multiple ways of rendering it inactive. Neural Cleanse [139] shows that it is possible to identify which neurons are strongly activated by the presence of the trigger and use them to detect inputs containing the trigger during testing. Similarly, they remove the influence of these neurons by pruning them. Additionally, Neural Cleanse fine-tunes the model to *unlearn* the trigger by adding it to clean samples and training with correct labels. Rather than reconstructing a single trigger, the authors of [140], [150], and

[151] model a distribution of possible triggers using a GAN, which is then sampled to train an immunized model.

4.3.2 Trigger-Agnostic Backdoor Removal

Another way to remove backdoor behavior is to modify the model, only keeping the parts necessary for the intended tasks. Since the backdoor is not active during forward passes on clean data, the backdoor behavior can be removed during this process. [109] attempts such a defense by pruning neurons that are dormant—i.e., they are not activated on clean inputs. Specifically, their defense tests the poisoned model with clean inputs, records the average activation of each neuron, and iteratively prunes neurons in increasing order of average activation. However, this pruning defense cannot remove the backdoor without significantly degrading performance [109].

To overcome the performance loss of pruning defenses alone, several works propose fine-tuning the model on a clean dataset [109], [152], [153]. Since the dataset does not include the backdoor trigger, the backdoor behavior may eventually be forgotten after updating the parameters during fine-tuning. Combining pruning with fine-tuning can indeed remove backdoors while preserving the overall model accuracy, even when an adversary crafts pruning-aware attacks [109]. However, if the dataset used for fine-tuning is small, model performance may suffer significantly [152].

To better preserve the model's accuracy on clean data, the watermark removal framework REfiT [152] leverages elastic weight consolidation [154]. This defensive training process slows down the learning of model weights that are relevant to the main prediction task while updating other weights that are likely responsible for memorizing watermarks. Similarly, WILD [153] includes a feature distribution alignment scheme to achieve a similar goal.

4.4 Preventing Poisoning During Training

The methods described above aim to either detect or fix an already poisoned model. Here, we describe training-time strategies to avoid poisoning in the first place. Most of these defenses are agnostic to the exact poisoning attack and are hence applicable to both the training-only and the backdoor setting.

4.4.1 Robust Statistics

At a high level, the goal of estimating statistical properties of the data in the presence of (adversarially placed) outliers is at the core of a field known as *robust statistics* [155], [156], going all the way back to the work of [157] and [158]. A significant amount of work in this space shows that, from an information-theoretic point of view, this problem is indeed tractable for a wide variety of tasks and data distributions [157], [159], [160], [161], [162], [163]. However, from a computational perspective, most of these approaches do not provide efficient implementations for high-dimensional datasets (which, after all, are at the core of modern ML).

Recently, there has been a flurry of activity focused on designing computationally efficient algorithms in this setting. [164] presents the first algorithm for learning a linear classifier under adversarial noise. More recently, [134] and [135] develop efficient algorithms for learning a number of

parametric distributions even when a fraction of the data has been arbitrarily corrupted. These algorithms rely on relatively simple primitives and can thus be efficiently implemented even for high-dimensional distributions [165]. In an orthogonal direction, [166] draws a connection between robust estimation and GANs [105] that allows one to approximate complex robust estimators efficiently [167].

While these algorithms focus on estimating specific statistical quantities of the data, other work optimizes general notions of risk in the presence of outliers. For instance, [168] propose algorithms for risk minimization based on recovering a list of possible models and (optionally) using a small uncorrupted dataset to choose between them. We refer the reader to [169], [170], and [171] for additional references on this line of work.

4.4.2 Randomized Smoothing

Randomized smoothing [172], [173] was originally proposed to defend against evasion attacks [86]. Starting with a base model, a smoothed version of that model is defined by replacing the model prediction on each data point by the majority prediction in its neighborhood. The outputs of the smoothed model can be computed efficiently, while at the same time certifying its robustness to input perturbations [173].

In the context of data poisoning, the goal is to protect the model against perturbations to the training set. Thus, the goal of *certifiable robustness* is for each test point, to return a prediction as well as a certificate that the prediction would not change had some quantity of training data been modified. To do so, one can view the entire training-plus-single-prediction pipeline as a function which can be robustified against input perturbations [62], [174]. Weber *et al.* [174] apply this defense to backdoor attacks, where backdoor pixel perturbations are in the continuous space. On the other hand, [62] uses this defense against label flipping attacks where label perturbations are in the discrete space.

4.4.3 Majority Vote Mechanisms

A number of approaches utilize majority vote mechanisms to ignore poisoned samples. The underlying assumption is that the number of poisoned samples injected by the attacker is small compared to the size of the overall training dataset. Therefore, the poison samples will not significantly influence the majority vote when voters each use only a subset of the data. For example, Deep Partition Aggregation [175] learns multiple base classifiers by partitioning the training set into disjoint subsets. Similarly, [176] train multiple base models on random subsamples of the training dataset. The base models are then combined via majority vote to produce an aggregate model, the robustness of which can be confirmed empirically and certified theoretically. Jia *et al.* [177] predict the label of a test example via majority vote among the labels of its k nearest neighbors or all of its neighbors within radius r in the training dataset.

4.4.4 Differential Privacy

Differential Privacy (DP) [178] was originally designed to protect the privacy of individuals contributing data. The core idea is that if the output of the algorithm remains

essentially unchanged when one individual input point is added or subtracted, the privacy of each individual is preserved. From the perspective of data poisoning, differential privacy ensures that model predictions do not depend too much on individual data points. Thus, models will not be disproportionately affected by poisoned samples. [19] studies defenses based on DP against data poisoning from the practical and theoretical perspectives. [179] empirically shows that the off-the-shelf mechanism DP-SGD [180], which clips and noises gradients during training, can serve as a defense. They point out that the main artifacts of gradients computed in the presence of poisoning are that their ℓ_2 -norms have higher magnitudes and their orientation differs from clean gradients. Since DP-SGD bounds gradient magnitudes by clipping and minimizes the difference in orientation by random noise addition, it is successful in defending against poisoning attacks [179]. A recent line of work proves that data augmentations which yield strong empirical defense against poisoning and backdoor attacks also enhances differential privacy guarantees all without degrading model performance [181], [182].

4.4.5 Input Preprocessing

Additionally, some works propose modifying the model input, during training or testing. Liu *et al.* [110] utilize an autoencoder [183] trained on clean data to preprocess the input. Since the input is not perfectly reconstructed (especially given that the autoencoder is not particularly sensitive to the trigger), the model is unlikely to recognize the trigger. [181] and [182] propose using strong data augmentations during training, such as *mixup* [184], CutMix [185], and MaxUp [186]. Dramatic data augmentations sufficiently disrupt triggers and perturbations to the training data, foiling the attack. This approach is highly effective against both training-only and backdoor attacks and has the added benefit that it does not degrade model performance [181], [182]. Adversarial training, which is typically used to defend against evasion attacks, has also been adapted to defend against training-only and backdoor train-time attacks [187]. This work trains against intentionally poisoned data in order to desensitize a model to dataset manipulations.

4.5 Defenses for Federated Learning

In Federated Learning (FL), a global model is trained by utilizing local data from many clients, providing a venue for new poisoning attacks from malicious clients. Since this setting allows for fundamentally different attacks compared to other learning settings (cf. Section 2.7), a number of application-specific defenses have been developed for FL. These include robust federated aggregation algorithms, robust federated training protocols, and post-training measures.

4.5.1 Robust Federated Aggregation

Robust federated aggregation algorithms attempt to nullify the effects of attacks while aggregating client updates. These methods can be broadly classified into two types; one type identifies and down-weights the malicious updates, while a second type does not attempt to identify malicious clients and instead computes aggregates in a way which is resistant to poisons. A prototypical idea for this second method

estimates a true “center” of the received model updates rather than taking a weighted average.

Fung *et al.* [72] identify sybils-based attacks, including label flipping and backdoors, as group actions. Sybils share a common adversarial objective whose updates are more similar to each other than honest clients. They propose Fool-sGold, which calculates the cosine similarity of the gradient updates from clients, reduces aggregation weights of clients that contribute similar gradient updates, thus promoting contribution diversity. Another avenue for defense in the FL setting is learning-based robust aggregation. Specifically, [188] utilizes a variational autoencoder (VAE) [189] to detect and remove malicious model updates in aggregation, where the VAE is trained on clean model updates. The encoder projects model updates into low-dimensional embeddings in latent space, and the decoder reconstructs the sanitized model updates and generates a reconstruction error. The idea is that the low-dimensional embeddings retain essential features, so malicious updates are sanitized and trigger much higher reconstruction errors while benign updates are unaffected. Updates with higher reconstruction errors are deemed malicious and are excluded from aggregation. This method assumes that a clean dataset is available to train the detection model.

Another approach involves Byzantine-robust aggregation techniques which resist manipulation, even without identifying malicious clients. Two such algorithms, called Krum and Multi-Krum [190], select representative gradient updates which are close to their nearest neighbor gradients. Another algorithm called Bulyan [191] first uses another aggregation rule such as Krum to iteratively select benign candidates and then aggregates these candidates by a variant of the trimmed mean [192].

Another class of approaches employs the coordinate-wise median gradient [192], geometric median of means [193], and approximate geometric median [194] since median-based computations are more resistant to outliers than mean-based aggregation. These methods are effective in robust distributed learning where the data of each user comes from the same distribution. However, they are less effective in FL where local data may be collected in a non-identically distributed manner across clients. Also, these methods are shown, both theoretically and empirically, to be less effective in the high-dimensional regime [191]. Another Byzantine-robust method called RSA [195] penalizes parameter updates which move far away from the previous parameter vector and provides theoretical robustness guarantees. Unlike earlier methods, RSA does not assume that all workers see i.i.d. data from the same distribution [195]. Alternatively, [196] estimates a regression line by a repeated median estimator [197] for each parameter dimension of the model updates, then dynamically assign aggregation weights to clients based on the residual of their model updates to that regression line.

4.5.2 Robust Federated Training

In addition to robust federated aggregation, several FL protocols mitigate poisoning attacks during training. Sun *et al.* [114] show that clipping the norm of model updates and adding Gaussian noise can mitigate backdoor attacks that are based on the model replacement paradigm [30], [113].

This work highlights the success of this method on the realistic federated EMNIST dataset [198], [199].

[200] leverages two specific features of federated learning to defend against backdoor attacks. Specifically, they utilize global models produced in the previous rounds and the fact that the attacker does not have access to a substantial amount of training data. They propose BaFFLe [200], which incorporates an additional validation phase to each round of FL. That is, a set of randomly chosen clients validate the current global model by computing a validation function on their private data and report whether the current global model is poisoned. The server decides to accept or reject the global model based on the feedback from validating clients. Specifically, the validation function compares the class-specific misclassification rates of the current global model with those of the accepted global models in the previous rounds and raises a warning when the misclassification rates differ significantly, which may indicate backdoor poisoning.

4.5.3 Post-Training Defenses

Other defense strategies focus on restoring the poisoned global model after training. Wu *et al.* [201] extend pruning and fine-tuning methods [109] to the FL setting to repair backdoored global models. Their method requires clients to rank the dormant level of neurons in the neural network using their local data, since the server itself cannot access clean training data. They then select and remove the most redundant neurons via majority vote [201].

4.6 Open Problems

- *Defenses beyond image classification:* Although data poisoning and backdoor attacks have been applied in a variety of domains, image classification remains to be the major focus of research for defenses. It is thus crucial that these defenses are applied to other domains in order to understanding potential for real-world use, as well as any shortcomings.
- *Navigating trade-offs between accuracy, security, and data privacy:* Modern large-scale ML systems strive to achieve high accuracy while maintaining user privacy. However, these goals seem to be at odds in the presence of data poisoning. In fact, most FL defenses rely on direct access to model updates, which can reveal information about user data [202], [203]. Achieving security against poisoning while maintaining accuracy and privacy appears to be elusive given our current methods.
- *Can defenses be bypassed without access to training?* [204] shows that one can bypass certain outlier-based defenses by enforcing that the internal representations of a model on poison examples during training are similar to those corresponding to clean examples. The open question is whether these defenses can be bypassed *without access to the training protocol*.
- *Efficient and practical defenses:* Many approaches to identifying poisoned models require producing a set of auxiliary clean and poisoned models to train a detector [144], [145], [146], but this process is computationally expensive. Moreover, generating additional models for trigger-agnostic methods or reconstructing possible

triggers in trigger reconstruction methods [139], [141] requires a clean dataset, which may not be feasible in practice. Therefore, designing efficient and practical defense methods with less data and computation requirements is essential for practical application.

- *Differential privacy and data poisoning:* [179] and [205] show that there remains a massive gap between the theoretical worst-case lower bounds provided by DP mechanisms and the empirical performance of the defenses against data poisoning. It is however unclear if this gap is due to existing attacks being insufficient or due to the theoretical bounds being unnecessarily pessimistic.
- *Certified defenses against poisoning attacks:* Certified defenses against poisoning attacks are still far from producing meaningful guarantees in realistic, large-scale settings. Moreover, they are particularly hard to study in a federated learning setting. Instead of analyzing the influence of training datasets on model predictions in an end-to-end manner like in the centralized setting [62], [174], one needs to consider how the local training datasets influence the local updates, and how the performance of the global model is influenced by these local updates through aggregation.
- *Detection of inconspicuous poison examples:* Defense strategies rely on poison examples or backdoor model behavior being noticeably irregular in the context of the ambient dataset. Detecting malicious behavior when it does not appear anomalous is a much harder task, and existing methods often fail. Similarly, anomaly detection is ineffective in federated learning, where each client may have a dramatically different underlying data distribution. Picking out malicious clients from benign, yet atypical ones is an important open problem.

5 CONCLUSION

The expansion of the power and scale of machine learning models has been accompanied by a corresponding expansion of their vulnerability to attacks. In light of the rapid surge in work on data poisoning and defenses against it, we provide a birds-eye perspective by systematically dissecting the wide range of research directions within this space. The open problems we enumerate reflect the interests and experience of the authors and are not an exhaustive list of outstanding research problems. Moving forward, we anticipate work not only on these open problems but also on datasets and benchmarks for comparing existing methods since controlled comparisons are currently lacking in the data poisoning and backdoor literature. We hope that the overhead perspective we offer helps to illuminate both urgent security needs within industry and also a need to understand security vulnerabilities so that the community can move towards closing them.

REFERENCES

- [1] J. Wakefield, "Microsoft chatbot is taught to swear on Twitter," *BBC News*, 2016. [Online]. Available: <https://www.bbc.com/news/technology-35890188>

- [2] Anonymous, "Possible malware found hidden inside images from the ImageNet dataset," 2020. [Online]. Available: https://www.reddit.com/r/MachineLearning/comments/j4jrln/d_possible_malware_found_hidden_inside_images/
- [3] B. Nelson *et al.*, "Exploiting machine learning to subvert your spam filter," in *Proc. 1st USENIX Workshop Large-Scale Exploits Emergent Threats*, 2008, pp. 1–9.
- [4] R. S. S. Kumar *et al.*, "Adversarial machine learning—industry perspectives," 2020, *arXiv:2002.05646*.
- [5] X. Yuan, P. He, Q. Zhu, and X. Li, "Adversarial examples: Attacks and defenses for deep learning," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 30, no. 9, pp. 2805–2824, Sep. 2019.
- [6] L. Lyu, H. Yu, and Q. Yang, "Threats to federated learning: A survey," 2020, *arXiv:2003.02133*.
- [7] Y. Li, B. Wu, Y. Jiang, Z. Li, and S.-T. Xia, "Backdoor learning: A survey," 2020, *arXiv:2007.08745*.
- [8] S. Kaviani and I. Sohn, "Defense against neural trojan attacks: A survey," *Neurocomputing*, vol. 424, pp. 651–667, 2021.
- [9] Y. Gao *et al.*, "Backdoor attacks and countermeasures on deep learning: A comprehensive review," 2020, *arXiv:2007.10760*.
- [10] N. Papernot, P. McDaniel, A. Sinha, and M. P. Wellman, "Sok: Security and privacy in machine learning," in *Proc. IEEE Eur. Symp. Secur. Privacy*, 2018, pp. 399–414.
- [11] N. Pitropakis, E. Panaousis, T. Giannetos, E. Anastasiadis, and G. Loukas, "A taxonomy and survey of attacks against machine learning," *Comput. Sci. Rev.*, vol. 34, 2019, Art. no. 100199.
- [12] H. Aghakhani *et al.*, "VENOMAVE: Clean-label poisoning against speech recognition," *CoRR*, vol. abs/2010.10682, 2020.
- [13] D. Solans, B. Biggio, and C. Castillo, "Poisoning attacks on algorithmic fairness," 2020, *arXiv:2004.07401*.
- [14] T. A. Meyer and B. Whateley, "SpamBayes: Effective open-source, Bayesian based, email classification system," in *Proc. 1st Conf. Email Anti-Spam*, 2004.
- [15] B. Li, Y. Wang, A. Singh, and Y. Vorobeychik, "Data poisoning attacks on factorization-based collaborative filtering," in *Proc. 30th Int. Conf. Neural Informat. Process. Syst.*, 2016, pp. 1893–1901.
- [16] M. Fang, G. Yang, N. Z. Gong, and J. Liu, "Poisoning attacks to graph-based recommender systems," in *Proc. 34th Annu. Comput. Secur. Appl. Conf.*, 2018, pp. 381–392.
- [17] R. Hu, Y. Guo, M. Pan, and Y. Gong, "Targeted poisoning attacks on social recommender systems," in *Proc. IEEE Glob. Commun. Conf.*, 2019, pp. 1–6.
- [18] M. Fang, N. Z. Gong, and J. Liu, "Influence function based data poisoning attacks to top-n recommender systems," in *Proc. Web Conf.*, 2020, pp. 3019–3025.
- [19] Y. Ma, X. Zhu, and J. Hsu, "Data poisoning against differentially-private learners: Attacks and defenses," in *Proc. 28th Int. Joint Conf. Artif. Intell.*, 2019, pp. 4732–4738.
- [20] S. Wadhwa, S. Agrawal, H. Chaudhari, D. Sharma, and K. Achan, "Data poisoning attacks against differentially private recommender systems," in *Proc. 43rd Int. ACM SIGIR Conf. Res. Develop. Informat. Retrieval*, 2020, pp. 1617–1620.
- [21] X. Cao, J. Jia, and N. Z. Gong, "Data poisoning attacks to local differential privacy protocols," in *Proc. 30th USENIX Secur. Symp.*, 2019, pp. 947–964.
- [22] C. Miao, Q. Li, H. Xiao, W. Jiang, M. Huai, and L. Su, "Towards data poisoning attacks in crowd sensing systems," in *Proc. 18th ACM Int. Symp. Mobile Ad Hoc NetWork. Comput.*, 2018, pp. 111–120.
- [23] Y. Ma, K.-S. Jun, L. Li, and X. Zhu, "Data poisoning attacks in contextual bandits," in *Proc. Int. Conf. Decis. Game Theory Secur.*, 2018, pp. 186–204.
- [24] F. Liu and N. Shroff, "Data poisoning attacks on stochastic bandits," in *Proc. Int. Conf. Mach. Learn.*, 2019, pp. 4042–4050.
- [25] X. Zhang, X. Zhu, and L. Lessard, "Online data poisoning attacks," in *Proc. Conf. Learn. Dyn. Control*, 2020, pp. 201–210.
- [26] S. Shan, E. Wenger, J. Zhang, H. Li, H. Zheng, and B. Y. Zhao, "Fawkes: Protecting personal privacy against unauthorized deep learning models," in *Proc. USENIX Secur. Symp.*, 2020, pp. 1589–1604.
- [27] I. Evtimov, P. Sturmfels, and T. Kohno, "FoggySight: A scheme for facial lookup privacy," 2020, *arXiv:2012.08588*.
- [28] V. Cherepanova *et al.*, "Lowkey: Leveraging adversarial attacks to protect social media users from facial recognition," 2021, *arXiv:2101.07922*.
- [29] C. Gao, V. Chandrasekaran, K. Fawaz, and S. Jha, "Face-Off: Adversarial face obfuscation," 2020, *arXiv:2003.08861*.
- [30] A. N. Bhagoji, S. Chakraborty, P. Mittal, and S. Calo, "Analyzing federated learning through an adversarial lens," in *Proc. Int. Conf. Mach. Learn.*, 2019, pp. 634–643.
- [31] G. Sun, Y. Cong, J. Dong, Q. Wang, and J. Liu, "Data poisoning attacks on federated machine learning," 2020, *arXiv:2004.10020*.
- [32] V. Tolpegin, S. Truex, M. E. Gursoy, and L. Liu, "Data poisoning attacks against federated learning systems," in *Proc. Eur. Symp. Res. Comput. Secur.*, 2020, pp. 480–501.
- [33] A. Shafahi *et al.*, "Poison frogs! Targeted clean-label poisoning attacks on neural networks," in *Proc. Adv. Neural Informat. Process. Syst.*, 2018, pp. 6103–6113.
- [34] A. Schwarzschild, M. Goldblum, A. Gupta, J. P. Dickerson, and T. Goldstein, "Just how toxic is data poisoning? A unified benchmark for backdoor and data poisoning attacks," 2020, *arXiv:2006.12557*.
- [35] J. Guo and C. Liu, "Practical poisoning attacks on neural networks," in *Proc. Eur. Conf. Comput. Vis.*, 2020, pp. 142–158.
- [36] C. Zhu *et al.*, "Transferable clean-label poisoning attacks on deep neural nets," 2019, *arXiv:1905.05897*.
- [37] H. Aghakhani, D. Meng, Y.-X. Wang, C. Kruegel, and G. Vigna, "Bullseye polytope: A scalable clean-label poisoning attack with improved transferability," 2020, *arXiv:2005.00191*.
- [38] Y. Ji, X. Zhang, S. Ji, X. Luo, and T. Wang, "Model-reuse attacks on deep learning systems," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2018, pp. 349–363.
- [39] L. Muñoz-González *et al.*, "Towards poisoning of deep learning algorithms with back-gradient optimization," in *Proc. 10th ACM Workshop Artif. Intell. Secur.*, 2017, pp. 27–38.
- [40] B. Biggio, B. Nelson, and P. Laskov, "Poisoning attacks against support vector machines," 2012, *arXiv:1206.6389*.
- [41] H. Xiao, B. Biggio, G. Brown, G. Fumera, C. Eckert, and F. Roli, "Is feature selection secure against training data poisoning?," in *Proc. Int. Conf. Mach. Learn.*, 2015, pp. 1689–1698.
- [42] S. Mei and X. Zhu, "Using machine teaching to identify optimal training-set attacks on machine learners," in *Proc. Conf. Assoc. Advance. Artif. Intell.*, 2015, pp. 2871–2877.
- [43] P. W. Koh and P. Liang, "Understanding black-box predictions via influence functions," in *Proc. Int. Conf. Mach. Learn.*, 2017, pp. 1885–1894.
- [44] C. Burkard and B. Lagesse, "Analysis of causative attacks against SVMs learning from data streams," in *Proc. 3rd ACM Int. Workshop Secur. Privacy Anal.*, 2017, pp. 31–36.
- [45] M. Jagielski, A. Oprea, B. Biggio, C. Liu, C. Nita-Rotaru, and B. Li, "Manipulating machine learning: Poisoning attacks and countermeasures for regression learning," in *Proc. IEEE Symp. Secur. Privacy*, 2018, pp. 19–35.
- [46] W. R. Huang, J. Geiping, L. Fowl, G. Taylor, and T. Goldstein, "Metapoisn: Practical general-purpose clean-label data poisoning," 2020, *arXiv:2004.00225*.
- [47] J. Geiping *et al.*, "Witches' Brew: Industrial scale data poisoning via gradient matching," 2020, *arXiv:2009.02276*.
- [48] A. Krizhevsky, "Learning multiple layers of features from tiny images," Univ. Toronto, Toronto, ON, Canada, Tech. Rep. TR-2009, 2009.
- [49] Y. Le and X. Yang, "Tiny imagenet visual recognition challenge," Stanford Univ., Stanford, CA, USA, Course Project CS 231N, 2015.
- [50] L. Fowl *et al.*, "Preventing unauthorized use of proprietary data: Poisoning for secure dataset release," 2021, *arXiv:2103.02683*.
- [51] C. Yang, Q. Wu, H. Li, and Y. Chen, "Generative poisoning attack method against neural networks," 2017, *arXiv:1703.01340*.
- [52] L. Muñoz-González, B. Pfizner, M. Russo, J. Carnerero-Cano, and E. C. Lupu, "Poisoning attacks with generative adversarial nets," 2019, *arXiv:1906.07773*.
- [53] J. Shen, X. Zhu, and D. Ma, "TensorClog: An imperceptible poisoning attack on deep neural network applications," *IEEE Access*, vol. 7, pp. 41498–41506, 2019.
- [54] H. Huang, X. Ma, S. M. Erfani, J. Bailey, and Y. Wang, "Unlearnable examples: Making personal data unexploitable," in *Proc. Int. Conf. Learn. Representations*, 2021.
- [55] J. Feng, Q.-Z. Cai, and Z.-H. Zhou, "Learning to confuse: Generating training time adversarial data with auto-encoder," in *Proc. Adv. Neural Informat. Process. Syst.*, 2019, Art. no. 1074.
- [56] P. W. Koh, J. Steinhardt, and P. Liang, "Stronger data poisoning attacks break data sanitization defenses," 2018, *arXiv:1811.00741*.

- [57] S. Basu, P. Pope, and S. Feizi, "Influence functions in deep learning are fragile," 2020, *arXiv:2006.14651*.
- [58] B. Biggio, B. Nelson, and P. Laskov, "Support vector machines under adversarial label noise," in *Proc. Asian Conf. Mach. Learn.*, 2011, pp. 97–112.
- [59] M. Zhao, B. An, W. Gao, and T. Zhang, "Efficient label contamination attacks against black-box learning models," in *Proc. Int. Joint Conf. Artif. Intell.*, 2017, pp. 3945–3951.
- [60] R. Zhang and Q. Zhu, "A game-theoretic analysis of label flipping attacks on distributed support vector machines," in *Proc. 51st Annu. Conf. Informat. Sci. Syst.*, 2017, pp. 1–6.
- [61] A. Paudice, L. Muñoz-González, and E. C. Lupu, "Label sanitization against label flipping poisoning attacks," in *Proc. Joint Eur. Conf. Mach. Learn. Knowl. Discov. Databases*, 2018, pp. 5–15.
- [62] E. Rosenfeld, E. Winston, P. Ravikumar, and J. Z. Kolter, "Certified robustness to label-flipping attacks via randomized smoothing," 2020, *arXiv:2002.03018*.
- [63] L. G. Valiant, "Learning disjunction of conjunctions," in *Proc. Int. Joint Conf. Artif. Intell.*, 1985, pp. 560–566.
- [64] M. Kearns and M. Li, "Learning in the presence of malicious errors," *SIAM J. Comput.*, vol. 22, no. 4, pp. 807–837, 1993.
- [65] P. Austrin, K.-M. Chung, M. Mahmoody, R. Pass, and K. Seth, "On the impossibility of cryptography with tamperable randomness," in *Proc. Annu. Cryptol. Conf.*, 2014, pp. 462–479.
- [66] S. Mahloujifar and M. Mahmoody, "Blockwise p-tampering attacks on cryptographic primitives, extractors, and learners," in *Proc. Theory Cryptogr. Conf.*, 2017, pp. 245–279.
- [67] S. Mahloujifar, D. I. Diochnos, and M. Mahmoody, "Learning under -tampering attacks," in *Proc. 26th Int. Conf. Algorithmic Learn. Theory*, 2018, pp. 572–596.
- [68] S. Mahloujifar, D. I. Diochnos, and M. Mahmoody, "The curse of concentration in robust learning: Evasion and poisoning attacks from concentration of measure," in *Proc. AAAI Conf. Artif. Intell.*, 2019, pp. 4536–4543.
- [69] J. Zhang, J. Chen, D. Wu, B. Chen, and S. Yu, "Poisoning attack in federated learning using generative adversarial nets," in *Proc. 18th IEEE Int. Conf. Trust, Secur. Privacy Comput. Commun./13th IEEE Int. Conf. Big Data Sci. Eng.*, 2019, pp. 374–380.
- [70] M. Fang, X. Cao, J. Jia, and N. Gong, "Local model poisoning attacks to byzantine-robust federated learning," in *Proc. 29th USENIX Secur. Symp.*, 2020, pp. 1605–1622.
- [71] D. Cao, S. Chang, Z. Lin, G. Liu, and D. Sun, "Understanding distributed poisoning attack in federated learning," in *Proc. IEEE 25th Int. Conf. Parallel Distrib. Syst.*, 2019, pp. 233–239.
- [72] C. Fung, C. J. Yoon, and I. Beschastnikh, "Mitigating sybils in federated learning poisoning," 2018, *arXiv:1808.04866*.
- [73] S. Mahloujifar, M. Mahmoody, and A. Mohammed, "Data poisoning attacks in multi-party learning," in *Proc. Int. Conf. Mach. Learn.*, 2019, pp. 4274–4283.
- [74] X. Chen, C. Liu, B. Li, K. Lu, and D. Song, "Targeted backdoor attacks on deep learning systems using data poisoning," 2017, *arXiv:1712.05526*.
- [75] T. Gu, B. Dolan-Gavitt, and S. Garg, "BadNets: Identifying vulnerabilities in the machine learning model supply chain," 2017, *arXiv:1708.06733*.
- [76] M. Jagielski, G. Severi, N. P. Harger, and A. Oprea, "Subpopulation data poisoning attacks," 2020, *arXiv:2006.14026*.
- [77] G. Ramakrishnan and A. Albarghouthi, "Backdoors in neural models of source code," 2020, *arXiv:2006.06841*.
- [78] J. Dai, C. Chen, and Y. Li, "A backdoor attack against LSTM-based text classification systems," *IEEE Access*, vol. 7, pp. 138872–138878, 2019.
- [79] X. Chen, A. Salem, M. Backes, S. Ma, and Y. Zhang, "BadNL: Backdoor attacks against NLP models," 2020, *arXiv:2006.01043*.
- [80] L. Sun, "Natural backdoor attack on text data," 2020, *arXiv:2006.16176*.
- [81] Z. Zhang, J. Jia, B. Wang, and N. Z. Gong, "Backdoor attacks to graph neural networks," 2020, *arXiv:2006.11165*.
- [82] Z. Xi, R. Pang, S. Ji, and T. Wang, "Graph backdoor," 2020, *arXiv:2006.11890*.
- [83] G. Severi, J. Meyer, S. Coull, and A. Oprea, "Exploring backdoor poisoning attacks against malware classifiers," 2020, *arXiv:2003.01031*.
- [84] G. Lovisotto, S. Eberz, and I. Martinovic, "Biometric backdoors: A poisoning attack against unsupervised template updating," 2019, *arXiv:1905.09162*.
- [85] P. Kiourt, K. Wardega, S. Jha, and W. Li, "Trojdr: Evaluation of backdoor attacks on deep reinforcement learning," in *Proc. 57th ACM/IEEE Des. Automat. Conf.*, 2020, pp. 1–6.
- [86] B. Biggio *et al.*, "Evasion attacks against machine learning at test time," in *Proc. Joint Eur. Conf. Mach. Learn. Knowl. Discov. Databases*, 2013, pp. 387–402.
- [87] C. Szegedy *et al.*, "Intriguing properties of neural networks," 2013, *arXiv:1312.6199*.
- [88] S.-M. Moosavi-Dezfooli, A. Fawzi, O. Fawzi, and P. Frossard, "Universal adversarial perturbations," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2017, pp. 1765–1773.
- [89] F. Tramèr, N. Papernot, I. Goodfellow, D. Boneh, and P. McDaniel, "The space of transferable adversarial examples," 2017, *arXiv:1704.03453*.
- [90] X. Zhang, Z. Zhang, and T. Wang, "Trojaning language models for fun and profit," 2020, *arXiv:2008.00312*.
- [91] E. Wallace, T. Z. Zhao, S. Feng, and S. Singh, "Customizing triggers with concealed data poisoning," 2020, *arXiv:2010.12563*.
- [92] R. Schuster, C. Song, E. Tromer, and V. Shmatikov, "You auto-complete me: Poisoning vulnerabilities in neural code completion," 2020, *arXiv:2007.02220*.
- [93] S. Ding, Y. Tian, F. Xu, Q. Li, and S. Zhong, "Trojan attack on deep generative models in autonomous driving," in *Proc. Int. Conf. Secur. Privacy Commun. Syst.*, 2019, pp. 299–318.
- [94] A. Salem, Y. Sautter, M. Backes, M. Humbert, and Y. Zhang, "BAAAN: Backdoor attacks against autoencoder and gan-based machine learning models," 2020, *arXiv:2010.03007*.
- [95] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, and I. Sutskever, "Language models are unsupervised multitask learners," *OpenAI blog*, vol. 1, no. 8, 2019, Art. no. 9.
- [96] Z. Yang, N. Iyer, J. Reimann, and N. Virani, "Design of intentional backdoors in sequential models," 2019, *arXiv:1902.09972*.
- [97] Y. Wang, E. Sarkar, M. Maniatakos, and S. E. Jabari, "Stop-and-go: Exploring backdoor attacks on deep reinforcement learning-based traffic congestion control systems," 2020, *arXiv:2003.07859*.
- [98] Y. Adi, C. Baum, M. Cisse, B. Pinkas, and J. Keshet, "Turning your weakness into a strength: Watermarking deep neural networks by backdooring," in *Proc. 27th USENIX Secur. Symp.*, 2018, pp. 1615–1631.
- [99] J. Zhang *et al.*, "Protecting intellectual property of deep neural networks with watermarking," in *Proc. Asia Conf. Comput. Commun. Secur.*, 2018, pp. 159–172.
- [100] S. Li, M. Xue, B. Zhao, H. Zhu, and X. Zhang, "Invisible backdoor attacks on deep neural networks via steganography and regularization," *IEEE Trans. Dependable Secure Comput.*, vol. 18, no. 5, pp. 2088–2105, Sep./Oct. 2021.
- [101] E. Wenger, J. Passananti, Y. Yao, H. Zheng, and B. Y. Zhao, "Backdoor attacks on facial recognition in the physical world," 2020, *arXiv:2006.14580*.
- [102] E. Sarkar, H. Benkraouda, and M. Maniatakos, "FaceHack: Triggering backdoored facial recognition systems using facial characteristics," 2020, *arXiv:2006.11623*.
- [103] A. Turner, D. Tsipras, and A. Madry, "Label-consistent backdoor attacks," 2019, *arXiv:1912.02771*.
- [104] A. Saha, A. Subramanya, and H. Pirsaviash, "Hidden trigger backdoor attacks," 2019, *arXiv:1910.00033*.
- [105] I. Goodfellow *et al.*, "Generative adversarial nets," in *Proc. Adv. Neural Informat. Process. Syst.*, 2014, pp. 2672–2680.
- [106] Y. Liu *et al.*, "Trojaning attack on neural networks," in *Proc. Netw. Distrib. Syst. Secur. Symp.*, 2018.
- [107] R. Tang, M. Du, N. Liu, F. Yang, and X. Hu, "An embarrassingly simple approach for trojan attack in deep neural networks," in *Proc. 26th ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining*, 2020, pp. 218–228.
- [108] M. Sun, S. Agarwal, and J. Z. Kolter, "Poisoned classifiers are not only backdoored, they are fundamentally broken," 2020, *arXiv:2010.09080*.
- [109] K. Liu, B. Dolan-Gavitt, and S. Garg, "Fine-pruning: Defending against backdooring attacks on deep neural networks," in *Proc. Int. Symp. Res. Attacks, Intrusions, Defenses*, 2018, pp. 273–294.
- [110] Y. Liu, Y. Xie, and A. Srivastava, "Neural trojans," in *Proc. IEEE Int. Conf. Comput. Des.*, 2017, pp. 45–48.
- [111] Y. Yao, H. Li, H. Zheng, and B. Y. Zhao, "Latent backdoor attacks on deep neural networks," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2019, pp. 2041–2055.

- [112] S. Wang, S. Nepal, C. Rudolph, M. Grobler, S. Chen, and T. Chen, "Backdoor attacks against transfer learning with pre-trained deep learning models," 2020, *arXiv:2001.03274*.
- [113] E. Bagdasaryan, A. Veit, Y. Hua, D. Estrin, and V. Shmatikov, "How to backdoor federated learning," in *Proc. Int. Conf. Artif. Intell. Statist.*, 2020, pp. 2938–2948.
- [114] Z. Sun, P. Kairouz, A. T. Suresh, and H. B. McMahan, "Can you really backdoor federated learning?," 2019, *arXiv:1911.07963*.
- [115] G. Baruch, M. Baruch, and Y. Goldberg, "A little is enough: Circumventing defenses for distributed learning," in *Proc. Adv. Neural Informat. Process. Syst.*, 2019, pp. 8635–8645.
- [116] H. Wang *et al.*, "Attack of the tails: Yes, you really can backdoor federated learning," in *Proc. Adv. Neural Informat. Process. Syst.*, 2020, pp. 16070–16084.
- [117] C. Xie, K. Huang, P.-Y. Chen, and B. Li, "DBA: Distributed backdoor attacks against federated learning," in *Proc. Int. Conf. Learn. Representations*, 2020.
- [118] Y. Liu, Z. Yi, and T. Chen, "Backdoor attacks and defenses in feature-partitioned collaborative learning," 2020, *arXiv:2007.03608*.
- [119] C.-L. Chen, L. Golubchik, and M. Paolieri, "Backdoor attacks on federated meta-learning," 2020, *arXiv:2006.07026*.
- [120] B. D. Rouhani, H. Chen, and F. Koushanfar, "DeepSigns: A generic watermarking framework for IP protection of deep learning models," 2018, *arXiv:1804.00750*.
- [121] Y. Uchida, Y. Nagai, S. Sakazawa, and S. Satoh, "Embedding watermarks into deep neural networks," in *Proc. ACM Int. Conf. Multimedia Retrieval*, 2017, pp. 269–277.
- [122] Y. Liu, X. Chen, C. Liu, and D. Song, "Delving into transferable adversarial examples and black-box attacks," in *Proc. Int. Conf. Learn. Representations*, 2017.
- [123] K. Eykholt *et al.*, "Robust physical-world attacks on deep learning visual classification," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2018, pp. 1625–1634.
- [124] A. Athalye, L. Engstrom, A. Ilyas, and K. Kwok, "Synthesizing robust adversarial examples," in *Proc. Int. Conf. Mach. Learn.*, 2018, pp. 284–293.
- [125] M. M. Breunig, H.-P. Kriegel, R. T. Ng, and J. Sander, "LOF: Identifying density-based local outliers," in *Proc. ACM SIGMOD Int. Conf. Manage. Data*, 2000, pp. 93–104.
- [126] F. E. Grubbs, "Procedures for detecting outlying observations in samples," *Technometrics*, vol. 11, no. 1, pp. 1–21, 1969.
- [127] V. Barnett and T. Lewis, "Outliers in statistical data", *Wiley Series in Probability and Mathematical Statistics Applied Probability and Statistics*. Hoboken, NJ, USA: Wiley, 1984.
- [128] A. Boukerche, L. Zheng, and O. Alfandi, "Outlier detection: Methods, models, and classification," *ACM Comput. Surv.*, vol. 53, no. 3, pp. 1–37, 2020.
- [129] J. Steinhardt, P. W. W. Koh, and P. S. Liang, "Certified defenses for data poisoning attacks," in *Proc. Adv. Neural Informat. Process. Syst.*, 2017, pp. 3517–3529.
- [130] I. Diakonikolas, G. Kamath, D. Kane, J. Li, J. Steinhardt, and A. Stewart, "Sever: A robust meta-algorithm for stochastic optimization," in *Proc. Int. Conf. Mach. Learn.*, 2019, pp. 1596–1606.
- [131] A. Prasad, A. S. Suggala, S. Balakrishnan, and P. Ravikumar, "Robust estimation via robust gradient estimation," 2018, *arXiv:1802.06485*.
- [132] A. Paudice, L. Muñoz-González, A. Gyorgy, and E. C. Lupu, "Detection of adversarial training examples in poisoning attacks through anomaly detection," 2018, *arXiv:1802.03041*.
- [133] B. Tran, J. Li, and A. Madry, "Spectral signatures in backdoor attacks," in *Proc. Adv. Neural Informat. Process. Syst.*, 2018, pp. 8000–8010.
- [134] I. Diakonikolas, G. Kamath, D. Kane, J. Li, A. Moitra, and A. Stewart, "Robust estimators in high-dimensions without the computational intractability," *SIAM J. Comput.*, vol. 48, no. 2, pp. 742–864, 2019.
- [135] K. A. Lai, A. B. Rao, and S. Vempala, "Agnostic estimation of mean and covariance," in *Proc. IEEE 57th Annu. Symp. Found. Comput. Sci.*, 2016, pp. 665–674.
- [136] S. Ma and Y. Liu, "NIC: Detecting adversarial samples with neural network invariant checking," in *Proc. 26th Netw. Distrib. Syst. Secur. Symp.*, 2019.
- [137] B. Chen *et al.*, "Detecting backdoor attacks on deep neural networks by activation clustering," 2018, *arXiv:1811.03728*.
- [138] N. Peri *et al.*, "Deep K-NN defense against clean-label data poisoning attacks," 2019, *arXiv:1909.13374*.
- [139] B. Wang *et al.*, "Neural cleanse: Identifying and mitigating backdoor attacks in neural networks," in *Proc. IEEE Symp. Secur. Privacy*, 2019, pp. 707–723.
- [140] H. Chen, C. Fu, J. Zhao, and F. Koushanfar, "DeepInspect: A black-box trojan detection and mitigation framework for deep neural networks," in *Proc. 28th Int. Joint Conf. Artif. Intell.*, 2019, pp. 4658–4664. [Online]. Available: <https://doi.org/10.24963/ijcai.2019/647>
- [141] W. Guo, L. Wang, X. Xing, M. Du, and D. Song, "TABOR: A highly accurate approach to inspecting and restoring trojan backdoors in AI systems," 2019, *arXiv:1908.01763*.
- [142] M. Fredrikson, S. Jha, and T. Ristenpart, "Model inversion attacks that exploit confidence information and basic countermeasures," in *Proc. 22nd ACM SIGSAC Conf. Comput. Commun. Secur.*, 2015, pp. 1322–1333.
- [143] R. Wang, G. Zhang, S. Liu, P.-Y. Chen, J. Xiong, and M. Wang, "Practical detection of trojan neural networks: Data-limited and data-free cases," in *Proc. Eur. Conf. Comput. Vis.*, 2020, pp. 222–238.
- [144] X. Xu, Q. Wang, H. Li, N. Borisov, C. A. Gunter, and B. Li, "Detecting AI trojans using meta neural analysis," in *Proc. IEEE Symp. Secur. Privacy*, 2021, pp. 103–120.
- [145] S. Huang, W. Peng, Z. Jia, and Z. Tu, "One-pixel signature: Characterizing CNN models for backdoor detection," in *Proc. Eur. Conf. Comput. Vis.*, 2020, pp. 326–341.
- [146] S. Kolouri, A. Saha, H. Pirsiavash, and H. Hoffmann, "Universal litmus patterns: Revealing backdoor attacks in CNNs," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2020, pp. 301–310.
- [147] Y. Gao, C. Xu, D. Wang, S. Chen, D. C. Ranasinghe, and S. Nepal, "STRIP: A defence against trojan attacks on deep neural networks," in *Proc. 35th Annu. Comput. Secur. Appl. Conf.*, 2019, pp. 113–125.
- [148] E. Chou, F. Tramer, and G. Pellegrino, "SentiNet: Detecting localized universal attack against deep learning systems," in *Proc. IEEE Secur. Privacy Workshops*, 2020, pp. 48–54.
- [149] R. R. Selvaraju, M. Cogswell, A. Das, R. Vedantam, D. Parikh, and D. Batra, "Grad-CAM: Visual explanations from deep networks via gradient-based localization," in *Proc. IEEE Int. Conf. Comput. Vis.*, 2017, pp. 618–626.
- [150] X. Qiao, Y. Yang, and H. Li, "Defending neural backdoors via generative distribution modeling," in *Proc. Adv. Neural Informat. Process. Syst.*, 2019, pp. 14004–14013.
- [151] L. Zhu, R. Ning, C. Wang, C. Xin, and H. Wu, "GangSweep: Sweep out neural backdoors by GAN," in *Proc. 28th ACM Int. Conf. Multimedia*, 2020, pp. 3173–3181.
- [152] X. Chen *et al.*, "REFIT: A unified watermark removal framework for deep learning systems with limited data," 2019, *arXiv:1911.07205*.
- [153] X. Liu, F. Li, B. Wen, and Q. Li, "Removing backdoor-based watermarks in neural networks with limited data," 2020, *arXiv:2008.00407*.
- [154] J. Kirkpatrick *et al.*, "Overcoming catastrophic forgetting in neural networks," *Proc. Nat. Acad. Sci. USA*, vol. 114, no. 13, pp. 3521–3526, 2017.
- [155] P. J. Huber, *Robust Statistics*, vol. 523. Hoboken, NJ, USA: Wiley, 2004.
- [156] F. R. Hampel, E. M. Ronchetti, P. J. Rousseeuw, and W. A. Stahel, *Robust Statistics: The Approach Based on Influence Functions*, vol. 196. Hoboken, NJ, USA: Wiley, 2011.
- [157] J. W. Tukey, "A survey of sampling from contaminated distributions," *Contributions to Probability and Statistics*. Princeton, NJ, USA: Princeton Univ. 1960.
- [158] P. J. Huber, "Robust estimation of a location parameter," *Ann. Math. Statist.*, vol. 35, pp. 73–101, 1964.
- [159] D. L. Donoho and R. C. Liu, "The automatic robustness of minimum distance functionals," *Ann. Statist.*, vol. 16, pp. 552–586, 1988.
- [160] Y. Zuo and R. Serfling, "General notions of statistical depth function," *Ann. Statist.*, vol. 28, pp. 461–482, 2000.
- [161] M. Chen *et al.*, "Robust covariance and scatter matrix estimation under huber's contamination model," *Ann. Statist.*, vol. 46, pp. 1932–1960, 2018.
- [162] J. Steinhardt, M. Charikar, and G. Valiant, "Resilience: A criterion for learning in the presence of arbitrary outliers," *Proc. Innov. Theor. Comput. Sci. Conf.*, vol. 94, p. 45, 2018.
- [163] B. Zhu, J. Jiao, and J. Steinhardt, "Generalized resilience and robust statistics," 2019, *arXiv:1909.08755*.

- [164] A. R. Klivans, P. M. Long, and R. A. Servedio, "Learning halfspaces with malicious noise," *J. Mach. Learn. Res.*, vol. 10, no. 12, pp. 2715–2740, 2009.
- [165] I. Diakonikolas, G. Kamath, D. M. Kane, J. Li, A. Moitra, and A. Stewart, "Being robust (in high dimensions) can be practical," in *Proc. Int. Conf. Mach. Learn.*, 2017, pp. 999–1008.
- [166] C. Gao, J. Liu, Y. Yao, and W. Zhu, "Robust estimation and generative adversarial nets," 2018, *arXiv:1810.02030*.
- [167] C. Gao, Y. Yao, and W. Zhu, "Generative adversarial nets for robust scatter estimation: A proper scoring rule perspective," *J. Mach. Learn. Res.*, vol. 21, pp. 1–48, 2020.
- [168] M. Charikar, J. Steinhardt, and G. Valiant, "Learning from untrusted data," in *Proc. Symp. Theory Comput.*, 2017, pp. 47–60.
- [169] J. Z. Li, "Principled approaches to robust machine learning and beyond," Ph.D. dissertation, Dept. Elect. Eng. Comput. Sci., Massachusetts Inst. Technol., Cambridge, MA, USA, 2018.
- [170] J. Steinhardt, "Robust learning: Information theory and algorithms," Ph.D. dissertation, Stanford Univ., Stanford, CA, USA, 2018.
- [171] I. Diakonikolas and D. M. Kane, "Recent advances in algorithmic high-dimensional robust statistics," 2019, *arXiv:1911.05911*.
- [172] M. Lecuyer, V. Atlidakis, R. Geambasu, D. Hsu, and S. Jana, "Certified robustness to adversarial examples with differential privacy," in *Proc. IEEE Symp. Secur. Privacy*, 2019, pp. 656–672.
- [173] J. Cohen, E. Rosenfeld, and Z. Kolter, "Certified adversarial robustness via randomized smoothing," in *Proc. Int. Conf. Mach. Learn.*, 2019, pp. 1310–1320.
- [174] M. Weber, X. Xu, B. Karlas, C. Zhang, and B. Li, "RAB: Provable robustness against backdoor attacks," 2020, *arXiv:2003.08904*.
- [175] A. Levine and S. Feizi, "Deep partition aggregation: Provable defense against general poisoning attacks," 2020, *arXiv:2006.14768*.
- [176] J. Jia, X. Cao, and N. Z. Gong, "Intrinsic certified robustness of bagging against data poisoning attacks," 2020, *arXiv:2008.04495*.
- [177] J. Jia, X. Cao, and N. Z. Gong, "Certified robustness of nearest neighbors against data poisoning attacks," 2020, *arXiv:2012.03765*.
- [178] C. Dwork, F. McSherry, K. Nissim, and A. Smith, "Calibrating noise to sensitivity in private data analysis," in *Proc. Theory Cryptogr. Conf.*, 2006, pp. 265–284.
- [179] S. Hong, V. Chandrasekaran, Y. Kaya, T. Dumitraş, and N. Papernot, "On the effectiveness of mitigating data poisoning attacks with gradient shaping," 2020, *arXiv:2002.11497*.
- [180] M. Abadi et al., "Deep learning with differential privacy," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2016, pp. 308–318.
- [181] E. Borgnia et al., "Strong data augmentation sanitizes poisoning and backdoor attacks without an accuracy tradeoff," 2020, *arXiv:2011.09527*.
- [182] E. Borgnia et al., "DP-Instahide: Provably defusing poisoning and backdoor attacks with differentially private data augmentations," 2021, *arXiv:2103.02079*.
- [183] P. Vincent, H. Larochelle, Y. Bengio, and P.-A. Manzagol, "Extracting and composing robust features with denoising autoencoders," in *Proc. 25th Int. Conf. Mach. Learn.*, 2008, pp. 1096–1103.
- [184] H. Zhang, M. Cisse, Y. N. Dauphin, and D. Lopez-Paz, "mixup: Beyond empirical risk minimization," 2017, *arXiv:1710.09412*.
- [185] S. Yun, D. Han, S. J. Oh, S. Chun, J. Choe, and Y. Yoo, "CutMix: Regularization strategy to train strong classifiers with localizable features," in *Proc. IEEE Int. Conf. Comput. Vis.*, 2019, pp. 6023–6032.
- [186] C. Gong, T. Ren, M. Ye, and Q. Liu, "Maxup: A simple way to improve generalization of neural network training," 2020, *arXiv:2002.09024*.
- [187] J. Geiping, L. Fowl, G. Somepalli, M. Goldblum, M. Moeller, and T. Goldstein, "What doesn't kill you makes you robust (er): Adversarial training against poisons and backdoors," 2021, *arXiv:2102.13624*.
- [188] S. Li, Y. Cheng, W. Wang, Y. Liu, and T. Chen, "Learning to detect malicious clients for robust federated learning," 2020, *arXiv:2002.00211*.
- [189] D. P. Kingma and M. Welling, "Auto-encoding variational bayes," 2013, *arXiv:1312.6114*.
- [190] P. Blanchard, R. Guerraoui, J. Stainer et al., "Machine learning with adversaries: Byzantine tolerant gradient descent," in *Proc. Adv. Neural Informat. Process. Syst.*, 2017, pp. 119–129.
- [191] E. M. E. Mhamdi, R. Guerraoui, and S. Rouault, "The hidden vulnerability of distributed learning in byzantium," 2018, *arXiv:1802.07927*.
- [192] D. Yin, Y. Chen, K. Ramchandran, and P. Bartlett, "Byzantine-robust distributed learning: Towards optimal statistical rates," 2018, *arXiv:1803.01498*.
- [193] Y. Chen, L. Su, and J. Xu, "Distributed statistical machine learning in adversarial settings: Byzantine gradient descent," *Proc. ACM Meas. Anal. of Comput. Syst.*, vol. 1, no. 2, pp. 1–25, 2017.
- [194] K. Pillutla, S. M. Kakade, and Z. Harchaoui, "Robust aggregation for federated learning," 2019, *arXiv:1912.13445*.
- [195] L. Li, W. Xu, T. Chen, G. B. Giannakis, and Q. Ling, "Rsa: Byzantine-robust stochastic aggregation methods for distributed learning from heterogeneous datasets," in *Proc. AAAI Conf. Artif. Intell.*, 2019, pp. 1544–1551.
- [196] S. Fu, C. Xie, B. Li, and Q. Chen, "Attack-resistant federated learning with residual-based reweighting," 2019, *arXiv:1912.11464*.
- [197] A. F. Siegel, "Robust regression using repeated medians," *Biometrika*, vol. 69, no. 1, pp. 242–244, 1982.
- [198] G. Cohen, S. Afshar, J. Tapson, and A. Van Schaik, "EMNIST: Extending MNIST to handwritten letters," in *Proc. Int. Joint Conf. Neural Netw.*, 2017, pp. 2921–2926.
- [199] S. Caldas et al., "LEAF: A benchmark for federated settings," 2018, *arXiv:1812.01097*.
- [200] S. Andreina, G. A. Marson, H. Möllering, and G. Karame, "BaFFLe: Backdoor detection via feedback-based federated learning," 2020, *arXiv:2011.02167*.
- [201] C. Wu, X. Yang, S. Zhu, and P. Mitra, "Mitigating backdoor attacks in federated learning," 2020, *arXiv:2011.01767*.
- [202] L. Zhu, Z. Liu, and S. Han, "Deep leakage from gradients," in *Proc. Adv. Neural Informat. Process. Syst.*, 2019, pp. 14 774–14 784.
- [203] J. Geiping, H. Bauermeister, H. Dröge, and M. Moeller, "Inverting gradients—How easy is it to break privacy in federated learning?," 2020, *arXiv:2003.14053*.
- [204] T. J. L. Tan and R. Shokri, "Bypassing backdoor detection algorithms in deep learning," 2019, *arXiv:1905.13409*.
- [205] M. Jagielski, J. Ullman, and A. Oprea, "Auditing differentially private machine learning: How private is private SGD?," in *Proc. Adv. Neural Informat. Process. Syst.*, 2020, pp. 22205–22216.



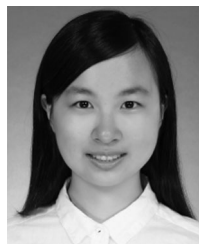
Micah Goldblum received the PhD degree in mathematics from the University of Maryland, where he is currently a postdoctoral research fellow under Professor Tom Goldstein. His research focuses on the mathematical underpinnings of deep neural networks, few-shot learning, and on building reliable and secure machine learning systems.



Dimitris Tsipras is currently working toward the PhD degree in computer science with the EECS department of MIT where he is advised by Aleksander Madry. His goal is to build reliable machine learning systems that can have an impact on the real world.



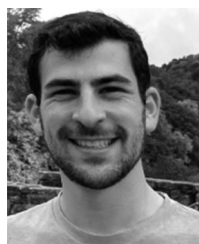
Chulin Xie is currently working toward the PhD degree with Computer Science Department, University of Illinois at Urbana-Champaign, advised by Prof. Bo Li. Her research interests include machine learning, adversarial robustness, federated learning and privacy.



Xinyun Chen is currently working toward the PhD degree with UC Berkeley, working with Prof. Dawn Song. Her research interests include the intersection of deep learning, programming languages, and security. Her recent research focuses on neural program synthesis, neural-symbolic reasoning, and adversarial machine learning. She received the Facebook Fellowship in 2020, and was selected for Rising Stars in EECS in 2020.



Aleksander Mądry received the PhD degree from MIT in 2011. He is currently a professor with Computer Science, MIT and leads the MIT Center for Deployable Machine Learning. Prior to joining the MIT faculty, he spent time with Microsoft Research New England and on the faculty of EPFL. His research interests span algorithms, continuous optimization, science of deep learning, and understanding machine learning from a robustness and deployability perspectives.



Avi Schwarzschild is currently working toward the PhD degree with the Applied Math and Scientific Computation program, the University of Maryland. He is advised by Tom Goldstein on his work in deep learning relating to data security and model vulnerability.



Bo Li is currently an assistant professor with the department of Computer Science, University of Illinois at Urbana–Champaign, and the recipient of the Symantec Research Labs Fellowship, Rising Stars, MIT Technology Review TR-35 Award, Intel Rising Star Award, Amazon Research Award, and best paper awards in several machine learning and security conferences. Her research focuses on both theoretical and practical aspects of security, machine learning, privacy, game theory, and adversarial machine learning.



Dawn Song received the PhD degree from UC Berkeley. She is currently a professor with the Department of Electrical Engineering and Computer Science, UC Berkeley. Her research interests lie in deep learning and security. She is the recipient of various awards including the MacArthur Fellowship, the Guggenheim Fellowship, the NSF CAREER Award, the Alfred P. Sloan Research Fellowship, and the MIT Technology Review TR-35 Award. She was an assistant professor with Carnegie Mellon University from 2002 to 2007.



Tom Goldstein received the PhD degree in mathematics from UCLA. He is currently the per-otto associate professor with Computer Science, University of Maryland. Prior to joining Maryland, he was a researcher with Stanford University and Rice University. His research focuses on optimization and scientific computing with applications to machine learning, computer vision, and natural language processing.

▷ **For more information on this or any other computing topic, please visit our Digital Library at www.computer.org/csdl.**